MaiAgent Technical Documentation

Table of Contents

1. Generative AI Quickstart

1.1 Large Language Models (LLM)

1.2 RAG Knowledge Base Retrieval System

1.3 Embedding Models

1.4 Reranker Models

1.5 Parser Tools

1.6 Image Recognition Support

2. Technical Principles

2.1 System Prompt

2.2 Response Evaluation and Monitoring Results

2.3 Text to SQL

2.4 Function Calling

2.5 Skill Modules

2.6 Canvas

2.7 AI Safety Guardrails

2.8 OAuth 2.0 Integration and Automatic Token Refresh

2.9 ICAP Content Validation Integration

3. MaiAgent Work｜Employee Productivity Integration

3.1 Add MaiGPT to Your System

3.2 Integrate MaiGPT: Choose the Right Agent and Add Capabilities

4. MaiAgent Serve｜Customer Service Integration

4.1 Software Vendor Integration Guide: Embed AI Agents for Your Customers

4.2 Web Chat Embedding and SDK

4.3 Embed MaiGPT Mode

4.4 Inject Page Context

4.5 LINE LIFF: What Is LINE LIFF

4.6 LINE LIFF: How to Integrate
4.7 Contact Overview and Integration
4.8 Contact Identity Synchronization and Token Updates
4.9 Differences Between Roles and Contacts
4.10 Third-Party Customer Service System Integration (Including Human Handoff)
4.11 Genesys Cloud Integration
4.12 Conversation Handoff and Continuation API
4.13 Webhook

5. MaiAgent Build｜API Integration Guide

5.1 Quickstart
5.2 Common API Conventions
5.3 Create Conversations and Messages
5.4 Conversations and Message Replies (Streaming/Synchronous)
5.5 Presigned File Upload Mode
5.6 Using the Knowledge Base—Upload, Search, and Manage
5.7 API Tool Engineering Guide
5.8 n8n Integration
5.9 Google Sheet Integration
5.10 Additional Endpoint Behaviors
5.11 Conversation Attachment Management API
5.12 Conversation Output List API
5.13 Conversation Visibility Permissions and Real-Time Events
5.14 Member Memory Management API
5.15 Site Branding Settings API
5.16 Credits Item Display Order
5.17 Agent Reply Modes and Structured Output Fields
5.18 Usage and Credits Fields in Conversation Records
5.19 Retirement of the Built-In Workflow Reply Mode

6. MaiAgent Build｜MCP and External Services

6.1 MCP (Model Context Protocol)
6.2 MaiAgent MCP: Let External AI Tools Operate the Platform
6.3 Self-Hosted MCP Server Integration Specification
6.4 Remote MCP Service Overview
6.5 Composio Integration

6.6 Zapier Integration

7. MaiAgent Build｜AI Gateway

7.1 AI Gateway: OpenAI-Compatible API Integration

7.2 AI Gateway Security

8. Identity and Access

8.1 Member Provisioning: SCIM / Invitations / SSO

8.2 Real-Time Synchronization: Call from User Events

8.3 Roster Synchronization Tools: CSV / JSON / Database

8.4 SSO and Identity Integration Overview

8.5 SAML SSO Integration

8.6 MaiAgent as an OAuth Authorization Server

8.7 Knowledge Management Permissions (Query Metadata) Overview

8.8 Start Building—Using JSON

9. Platform Setup and Deployment

9.1 Overview

9.2 Platform Environments (SaaS/Private Cloud/On-Premises)

9.3 Deployment Architecture

9.4 Deployment Types and Feature Comparison

9.5 System Requirements

9.6 Versions and Upgrades

9.7 On-Premises Model Service Integration

9.8 Operations: Offline Installation, TLS, Backups, Monitoring, and Acceptance

9.9 Bring Your Own Vector Database

9.10 Cloud Model Inference API Service

9.11 GPU Compute Hardware Planning

9.12 Online Office Editing in the File Library (ONLYOFFICE Docs Integration)

Generative AI Quickstart

Large Language Models (LLM)

Key Selection Criteria

Consider the following key factors when choosing a large language model:

Environment: determine whether to use cloud or on-premises models based on whether your environment can access the internet.

Quality: the model's ability to generate responses and follow instructions.

Speed: text generation speed and latency requirements to ensure adequate model response times.

Pricing: Credits pricing varies by model. Choose based on your usage needs and budget. See Credits Billing for pricing details.

Other factors: support for multimodal input and Function calling.

For comparisons of model quality, speed, and pricing, refer to continuously updated third-party evaluations such as Artificial Analysis.

Large Language Models Supported on MaiAgent

Models available on MaiAgent are continuously updated as providers release and retire models. This page lists providers and model families rather than specific model versions. For the models your organization can actually use, refer to the "Model Access Permissions" page in the admin console. This page lists the models MaiAgent has made available to your organization, and organization owners can enable or disable models within that selection.

Closed-Source Models

Model Provider	Model Family	Inference Services
OpenAI	GPT, o series	OpenAI, Azure, AWS Bedrock
Anthropic	Claude	AWS Bedrock, Google Vertex AI
Google	Gemini	Google Vertex AI
xAI	Grok	OpenAI-compatible API
Alibaba	Qwen (Max, Plus)	OpenAI-compatible API
Amazon	Nova	AWS Bedrock

Open-Source Models

Open-weight models can be used on MaiAgent SaaS through cloud inference services or deployed on self-hosted GPUs in a private cloud or on premises. For hardware requirements, see the GPU chapter.

Model Provider	Model Family	Inference Services on SaaS
OpenAI	GPT-OSS	OpenAI-compatible API
Google	Gemma	OpenAI-compatible API
Alibaba	Qwen, Qwen-VL	OpenAI-compatible API
DeepSeek	DeepSeek	AWS Bedrock, Azure
Mistral AI	Mistral	OpenAI-compatible API
Moonshot AI	Kimi	OpenAI-compatible API
IBM	Granite	OpenAI-compatible API

Models available for self-hosted on-premises deployment are determined by the model catalog in the MaiAgent deployment package, which currently includes families such as GPT-OSS, Gemma, Qwen, and Qwen-VL. Contact your MaiAgent representative for deployment planning.

> Each provider type supports a different range of LLM, Embedding, and Reranker models. See Model API Inference Services for details.

Do Models Always Need Fine-Tuning?

As AI technology advances rapidly, language models have developed strong language understanding and generation capabilities and are widely used across many fields. For example, pretrained language models can readily handle everyday conversations, article generation, and simple Q&A tasks.

However, when models face more challenging tasks in specialized fields such as medicine, law, or technical support, relying solely on pretrained models may not provide the best performance. Some developers choose **Fine-tuning**, which involves additional training for a specific domain to improve a model's specialized knowledge.

Fine-tuning is not the only solution. Two other effective approaches can achieve this goal: **Prompt Engineering** and **RAG (Retrieval-Augmented Generation)**.

1. Prompt Engineering: Optimize Model Performance with Precise Prompts

Prompt Engineering involves designing precise prompts to guide a model toward the desired results. The core approach is to formulate detailed instructions based on task requirements, helping the model narrow its response scope and understand the context and required output format.

Suppose a language model is intended to recommend suitable products based on a user's needs. A request such as “I want to buy a high-performance phone” may lead to an imprecise answer because “high-performance” can mean many things, such as processing speed, camera performance, or battery life.

To improve recommendation accuracy, we can use **Prompt Engineering** to design more specific questions or provide additional context that helps the model understand the user's needs. For example, use the following prompt instead:

"I need a phone with **long battery life** and an **efficient processor**, priced between **USD 500 and USD 800**. Please recommend a few phones that meet these criteria."

2. RAG: Improve Generation with External Knowledge

RAG improves model performance by combining external knowledge retrieval with the generation process.

In traditional generation tasks, models rely solely on knowledge learned during pretraining. RAG uses a retrieval system to obtain external information in real time and combines it with the generative model to answer questions or generate text more accurately.

For example, in medicine, when a model is asked about a rare disease, RAG can first retrieve relevant information from a specialized medical database and then use it to generate a more accurate answer. This approach allows the model to produce high-quality responses by retrieving existing knowledge even if it has never encountered that information during training. It is particularly useful when knowledge needs to be updated in real time and can greatly expand the model's knowledge coverage.

> For a more detailed introduction to RAG, see the next chapter, “RAG Knowledge Base Retrieval System Overview”.

Fine-tuning, Prompt Engineering, and RAG each have their own advantages and areas of application. Choose the most suitable strategy for your use case and requirements rather than relying solely on Fine-tuning.

Prompt Engineering offers a low-cost, flexible solution by designing precise prompts to guide models toward high-quality results.

RAG combines external knowledge with generation capabilities to provide more accurate answers when knowledge must be retrieved dynamically.

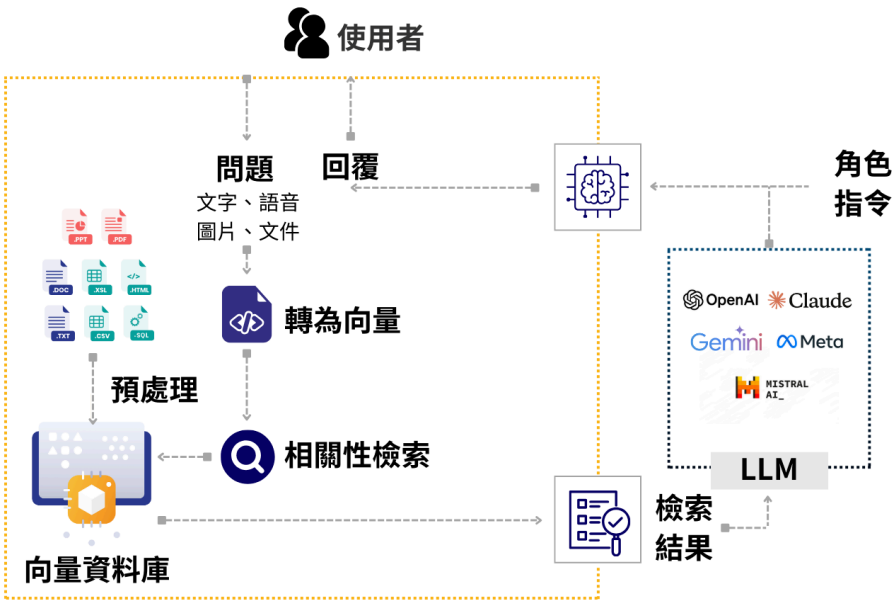
Fine-tuning can significantly improve model performance in specific domains, but requires substantial specialized data and additional resources. It can be considered a last resort when both **Prompt**

Engineering and **RAG** fail to meet the need.

> The MaiAgent platform does not provide model fine-tuning: the admin console has no interface for training or fine-tuning models, and the platform does not use your data to train any models. The Fine-tuning discussed above is performed outside the platform by model providers or self-hosted model teams. Models that have already been fine-tuned can be connected to the platform as follows: connect fine-tuned models hosted by cloud providers using your own API key (see AI Gateway: OpenAI-Compatible API Integration); deploy your own trained open-weight models as OpenAI-compatible model services in a private cloud or on premises and then connect them. See On-Premises Model Service Integration. To give an Agent specialized knowledge, start with System Prompts and a RAG Knowledge Base.

RAG Knowledge Base Retrieval System

RAG (Retrieval-Augmented Generation) lets a large language model retrieve passages relevant to a question from a knowledge base before answering based on those passages. This allows the model to reference the organization's own documents instead of relying solely on training data.



RAG workflow

Think of RAG as an open-book exam: the model does not need to memorize all the data. Before each answer, it looks through the knowledge base for the passages most relevant to the question and uses them as evidence. For this reason, it is best to include only data relevant to the Agent's tasks in the knowledge base. Irrelevant content may still be retrieved as context and make answers less accurate.

This page explains MaiAgent's actual processing workflow and configurable parameters. For instructions in the admin interface, see Knowledge Base Overview in the user manual.

How Data Enters the Knowledge Base

Files uploaded to the knowledge base go through these steps in order:

Parsing: A parser converts the file into text (including tables and images when needed). See Parser Tools.

Chunking: Text is split into passages (chunks), which serve as the units for retrieval and citations.

Embedding: The Embedding model selected for the knowledge base converts each passage into a vector. See Embedding Models.

Writing to the vector database: Passage text and vectors are written together to a vector database. The platform's built-in database is used by default, but you can connect your own. See Bring Your Own Vector Database.

Content becomes retrievable only after the file status changes to `done`.

The Workflow for a Single Retrieval

1. Determine the Retrieval Scope

Before retrieval, the platform determines which knowledge bases, files, and labels can be queried and applies these conditions directly to the vector database query. Passages outside this scope are never retrieved and therefore never reach the model.

The scope is taken in order from: message → conversation → contact's Query Metadata. If none of these is configured, the Agent's default label filter applies. If no filter is configured either, the entire knowledge base is queried. For setup instructions, see Query Metadata Overview and Knowledge Base Label Filtering in the user manual.

> To restrict who can see which documents, use Query Metadata or label filtering instead of relying solely on the system prompt. The system prompt is text provided to the model after retrieval and cannot block passages that have already been retrieved.

2. Hybrid Search

By default, **hybrid search** runs two types of queries simultaneously:

Vector search: Uses the question's vector to find semantically similar passages, even when the wording differs.

Keyword search (BM25): Matches words and terms, making it useful for product model numbers, proper nouns, and codes.

When Enable Reciprocal Rank Fusion (RRF) is turned on for the knowledge base, the two sets of results are merged by rank using RRF. Using RRF on Elasticsearch requires an Enterprise, Platinum, or Trial license. If your own Elasticsearch instance lacks such a license, the admin interface automatically disables this option. When hybrid search is disabled on the platform, only vector search is used.

Agent conversations always follow the platform's retrieval settings and cannot select a separate mode. However, Search Testing in the admin interface and the Knowledge Base Retrieval API let you specify vector-only (Semantic Search, `semantic`) or keyword-only (Keyword Search, `keyword`, Elasticsearch only) retrieval to compare what each method finds.

3. Reranking with Reranker

When Enable Search Result Reranking (Reranking) is turned on for the knowledge base, the first stage retrieves extra candidate passages (Number of Retrieved Chunks × the platform's configured multiplier, 7 by default). The Reranker model then compares the question with each passage, rescores them, and retains only the highest-scoring Number of Retrieved Chunks results. See Reranker Models.

If the organization's Credits fall below the platform threshold, that conversation skips Reranker and uses the first-stage results directly.

4. Similarity Threshold

The Similarity Threshold (0–1) filters out passages whose Reranker scores fall below the threshold. The default is 0, meaning no filtering. Because the threshold applies to Reranker scores, **it only takes effect when Reranker is enabled**. Setting it too high may remove every passage, leaving the model without reference material.

5. Merge Results from Multiple Knowledge Bases

When an Agent has multiple knowledge bases attached, their results are merged. The Agent's Maximum Number of Retrieved Blocks then determines how many passages are ultimately sent to the model. If any knowledge base queried in this retrieval has Reranker enabled, the merged results are reranked once more.

6. Send to the Model and Add Citations

The selected passages are sent to the large language model along with the Agent's system prompt. If Use During Retrieval is checked for a file metadata field, that field's value is also sent to the model as reference information for answering. Citations in the response link back to the source passages and files.

Agentic RAG: Retrieval in Agent Mode

The workflow above performs “one retrieval, one generation”: each message always triggers retrieval before an answer, and the model does not decide whether or what to search for. Agentic RAG lets the model decide how to retrieve. While answering, it can call retrieval multiple times, adjust queries based on earlier results, change keywords, or read another file before composing an answer.

In MaiAgent, these approaches correspond to the Agent's reply modes:

Reply mode	Retrieval method
RAG Q&A (`replyMode: normal` in the API)	Single retrieval: each message triggers one retrieval using the workflow on this page, and passages are sent directly to the model
Agent mode (`replyMode: agent`)	Agentic RAG: the knowledge base is provided to the model as tools. Within one turn, the model can retrieve passages multiple times, filter files by filename or metadata, search file contents by keyword, and read entire files. Each call follows the retrieval workflow on this page (including Query Metadata scope restrictions)

The number of retrieval rounds in Agent mode is constrained by the Agent's execution limits (see Function Calling: Execution Limits), and latency and usage are higher than with a single retrieval. RAG Q&A is sufficient for Q&A scenarios where the answer is in a single passage. Use Agent mode for scenarios that require comparing multiple documents or finding a file before reading it closely.

How Chat Mode and Agent Mode Query the Knowledge Base

Building on the previous section, the differences between the two modes are compared below:

	RAG Q&A (chat mode, `replyMode: normal`)	Agent mode (`replyMode: agent`)
When retrieval occurs	**Always** retrieves once before answering each message, without a model decision	Retrieval is a **tool**; the model decides whether to call it, how many times, and which query string to use
How many results are retrieved	Each knowledge base retrieves its own Number of Retrieved Chunks; merged results are truncated according to the Agent's Maximum Number of Retrieved Blocks	The model supplies `top_k` for the `retrieve_text_nodes` tool (default 3), so it usually retrieves fewer than the configured number of chunks. When Force Full Retrieval Scope is checked for the Agent, Maximum Number of Retrieved Blocks is always used instead
Available tools	None; only the passages retrieved this time	`retrieve_text_nodes` (semantic/hybrid retrieval), `query_files` (lists files and metadata), `grep_files` (matches strings in file contents), `read_file` (reads entire files), `get_surrounding_context` (retrieves surrounding passage context), `image_search` (finds images in multimodal knowledge bases; can be disabled in tool settings)
Query Metadata, label filtering, Reranker,	Apply to both modes under the same rules	

	RAG Q&A (chat mode, `replyMode: normal`)	Agent mode (`replyMode: agent`)
similarity threshold		
Attachments uploaded in the conversation	Parsed and indexed immediately on upload, then retrieved alongside the knowledge base when answering, using the Agent's maximum number of retrieved blocks	Not parsed on upload; the Agent uses attachment tools (`parse_attachment_content`, `read_attachment_full_text`, `ingest_attachment`) to read or index them according to the question. See How Message Attachments Are Processed
When search results are unsatisfactory	Adjust knowledge base parameters and the Agent's maximum number of retrieved blocks	First check whether the model called retrieval tools (inspect tool calls in Conversation Details and Execution Timeline), then adjust parameters or require knowledge base retrieval in the system prompt

When Enable Tool Search (dynamic tool search) is turned on for the Agent, knowledge base tools are also in the pool that must be searched before loading. Like Text to SQL and API tools, they require a call to tool_searching_tool first. The model may occasionally skip the search, answer that "no knowledge base retrieval tool is available," and use only other tools. If this happens, explicitly state in the system prompt: "Query the knowledge base with retrieve_text_nodes before answering," or disable tool search to make all tools directly available. Attaching a knowledge base does not automatically trigger retrieval, and you do not need to create a separate retrieval tool.

Configurable Parameters

Parameter	Setting location	Default	Description
Embedding Model	Knowledge base	Platform setting	Cannot be changed in the admin interface after creation; create another knowledge base or contact your MaiAgent representative to change it
Number of Retrieved Chunks	Knowledge base	12	Maximum number of passages retrieved from each knowledge base; minimum 1
Enable Search Result Reranking (Reranking)	Knowledge base	Enabled	Uses the Reranker model selected for the knowledge base when enabled
Similarity Threshold (0–1)	Knowledge base	0	Takes effect only when Reranker is enabled

Parameter	Setting location	Default	Description
Enable Reciprocal Rank Fusion	Knowledge base	Platform setting	Method for merging hybrid search results; must be enabled for hybrid search on OpenSearch
Maximum Number of Retrieved Blocks	Agent	12	Number of passages ultimately sent to the model during a conversation; the interface recommends 5–15, and higher values increase latency

To check which passages a question retrieves, use Search Testing in the admin interface or call the Knowledge Base Retrieval API.

How to Tune

Chunk size and overlap are determined by the parser and cannot be adjusted through fields in the admin interface. You can adjust the parameters below. Change only one at a time and compare results in Search Testing with the same set of questions.

Symptom	Check first	Then adjust
The correct passage is found but ranks too low or is truncated	Its ranking in Search Testing	Enable Reranker; if that is insufficient, increase the number of retrieved chunks and maximum number of retrieved blocks (latency and token usage will increase)
Irrelevant passages are frequently included	Passage order after enabling Reranker	Raise the similarity threshold (by 0.05–0.1 at a time; too high removes all results), or narrow the scope with label filtering
Model numbers, codes, or proper nouns cannot be found	Whether keyword search finds them	Ensure the document uses the same wording as the user; re-enable hybrid search if disabled (Elasticsearch requires an RRF license)
Answers cover only part of a table or long document	Whether chunks split the table	Reparse with a different parser, add table headers or section names within chunks, or enable Force Full Retrieval Scope for the Agent
Answers use only attachments and ignore the knowledge base (or vice versa)	Tool call logs in Agent mode	Explicitly require knowledge base retrieval first in the system prompt; chat mode retrieves both together, so increase the maximum number of retrieved blocks
Reranker is enabled but has no effect	Whether the organization's Credits are below the platform threshold (which automatically skips Reranker)	Top up Credits first, then compare

The Embedding model cannot be changed after creation; changing models means rebuilding the knowledge base. Compare results from different models in Search Testing before going live and making your final choice.

Embedding Models

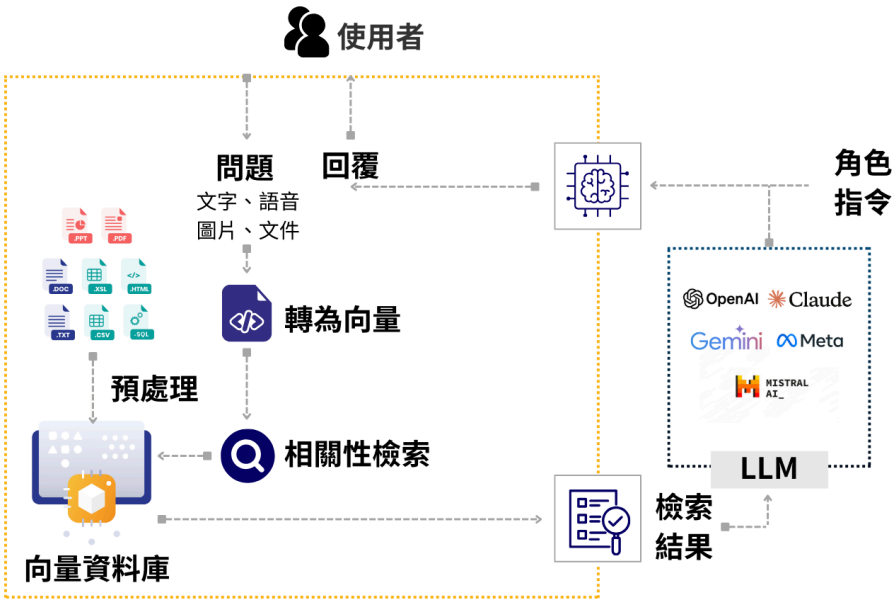
What Is Embedding?

An Embedding model converts text into a sequence of numbers (a vector). Text with similar meanings produces similar vectors, so the system can use vector distances to find semantically related passages, even when a question and a document use different wording.

In MaiAgent's RAG workflow, the Embedding model performs two tasks:

Build the index: Convert each passage from parsed knowledge base content into a vector and store it in the vector database.

Convert the question: Convert the user's question into a vector and compare its similarity with passages in the database.



RAG workflow

For the complete retrieval workflow, see RAG Knowledge Base Retrieval System.

What to Know Before Choosing

You cannot change it in the admin console after creation: You select the Embedding model when creating a knowledge base. Vectors produced by different models cannot be compared with each other. To

switch models, create a new knowledge base or contact your MaiAgent representative.

Image files require a multimodal model: To add image files (such as `.png` and `.jpg`) to a knowledge base and index them, the knowledge base must use an Embedding model that supports multimodal input. Otherwise, parsing those files will fail.

An Agent can use multiple models: A single Agent can connect to knowledge bases that use different Embedding models. The platform queries each knowledge base separately and combines the results.

Models Enabled in MaiAgent Cloud

The following models are enabled in the production environment as of October 2026. The models available when creating a knowledge base are those shown in the admin console.

Model (name in the admin console)	Provider	Supports image files
OpenAI text-embedding-3	OpenAI (`text-embedding-3-large`)	No
Cohere Embed V4.0	Cohere (`embed-v4.0`)	Yes
Cohere Embed Multilingual v3.0 (Bedrock)	AWS Bedrock (`cohere.embed-multilingual-v3`)	No

On-Premises Deployment

On-premises and private cloud deployments can run an Embedding model locally (the deployment package's model catalog includes `jina-embeddings-v3`, which requires a GPU), or connect to an existing model service or cloud provider. For available models and connection options, see Connecting Model Services for On-Premises Deployments. For Embedding support from each cloud provider, see Cloud Model Inference API Services.

Interface Requirements for Your Own Embedding Service

When connecting to a self-hosted or third-party OpenAI-compatible Embedding service, the URL must include `/v1`, and the service must accept the parameters sent by MaiAgent. Otherwise, parsing will fail for all knowledge base files. For the full requirements, see Connecting Model Services for On-Premises Deployments: Limitations for Your Own Embedding and Reranker Services.

> The Embedding model determines whether relevant content can be found. If a question frequently fails to retrieve the correct passage, first check the results using Search Test in the admin console. Then consider adjusting the Reranker, the number of retrieved passages, or rebuilding the knowledge base with a different Embedding model.

Reranker Models

What Is a Reranker?

A Reranker is a model designed specifically for reranking. First-stage retrieval (vector and keyword search) is fast but only provides an approximate match. The Reranker evaluates the relevance of each candidate passage to the question, then reorders the passages by score so that those that actually answer the question appear first.

Initial Retrieval

Quickly retrieve a set of candidate passages from the knowledge base using vector and keyword search.

Rescoring

The Reranker evaluates each question–passage pair and assigns a relevance score to each passage.

Selection

Keep the highest-scoring passages and filter out those below the similarity threshold, then pass them to the large language model.

How MaiAgent Uses Reranker

Each knowledge base selects its own Reranker model and controls it with the Enable Search Result Reranking (Reranking) toggle (enabled by default).

When enabled, the first stage retrieves candidates equal to the Number of Retrieved Passages × the platform's multiplier (7 by default). After reranking, only the Number of Retrieved Passages is retained.

The Similarity Threshold (0–1) applies to Reranker scores, so it only takes effect when Reranker is enabled.

When disabled, the first stage directly retrieves the Number of Retrieved Passages, without reranking or applying the similarity threshold.

For the complete workflow, see RAG Knowledge Base Retrieval System.

Models Enabled in MaiAgent Cloud

The following models are enabled in the production environment as of October 2026. Refer to the admin interface for the models available when creating a knowledge base.

Model (Admin Interface Name)	Provider
Cohere Rerank v3.5	Cohere (`rerank-v3.5`)
Cohere Rerank v4 Pro	Cohere (`rerank-v4.0-pro`)
Cohere Rerank v4 Fast	Cohere (`rerank-v4.0-fast`)

On-Premises Deployment

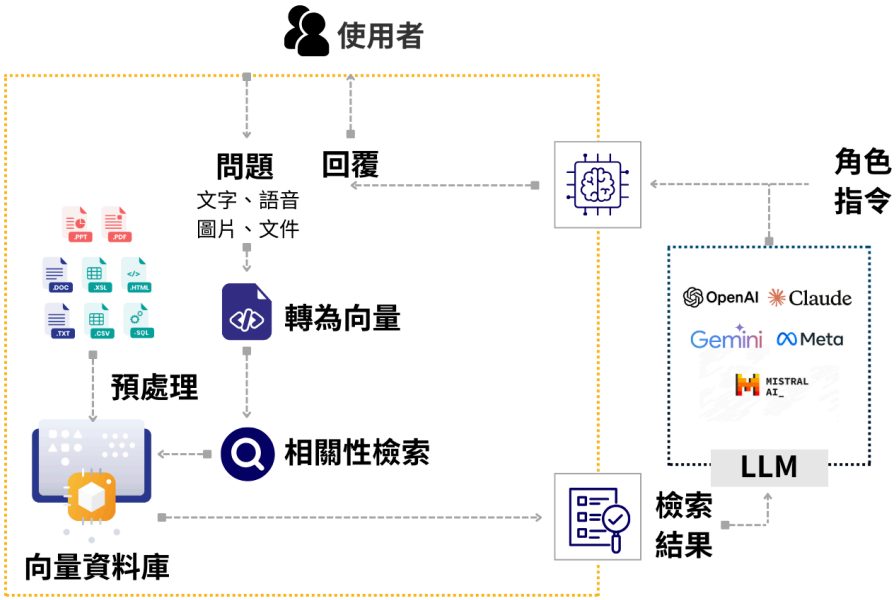
On-premises and private cloud environments can deploy a Reranker locally (the deployment package's model catalog provides `jina-reranker-v2-base-multilingual` and `bge-reranker-v2-m3`, which require a GPU), or connect to an existing model service or cloud provider. For available models and connection methods, see On-Premises Model Service Integration. For Reranker support from each cloud provider, see Cloud Model Inference API Services.

> Do not add /v1 to the URL when using your own Reranker service, or all knowledge base searches will fail. For URL rules, see On-Premises Model Service Integration: Connect to an Existing OpenAI-Compatible Model Service.

Parser Tools

What Is a Parser?

A parser converts raw files uploaded to the knowledge base into text before embedding and retrieval. Parsing quality directly determines whether content can be found later: this step determines whether tables retain their structure and whether text in images and scanned pages is extracted.



RAG workflow

You can specify a parser for each file during upload. For instructions in the admin interface, see How to Create a Knowledge Base: Basic Settings in the user manual. You can also call `GET /api/v1/allowed-file-specs/` to retrieve the file types supported by each parser.

Document Parsers

MaiAgent Cloud currently provides three document parsers:

Item	MaiAgent Parser (default)	MaiAgent Parser (Offline)	Vision Parser
Processing method	Reads text, tables, and images directly from the file	Uses layout analysis to preserve document	Uses a cloud multimodal model to visually read the

Item	MaiAgent Parser (default)	MaiAgent Parser (Offline)	Vision Parser
		structure (headings and tables)	document page by page and convert it to Markdown
Images and scanned pages	Extracts embedded images; performs OCR on vector drawing pages without a text layer	Adds OCR and image descriptions for PDFs when a multimodal model is configured	Uses visual understanding to read content in images, charts, and scanned pages
Uses a large language model	No	Only when a multimodal model is configured	Yes
Offline (no network) environments	Supported	Supported; image descriptions require an on-premises multimodal model	Not supported; requires a connection to a cloud model
Supported formats	`.pdf` `.doc` `.docx` `.odt` `.xls` `.xlsx` `.csv` `.ods` `.ppt` `.pptx` `.odp` `.epub` `.txt` `.json` `.jsonl` `.md` `.eml` and image files `.png` `.jpg` `.jpeg` `.gif` `.webp` `.tiff`	`.pdf` `.docx` `.pptx` `.md` `.html` `.htm`	`.pdf` `.doc` `.docx` `.odt` `.ppt` `.pptx` `.odp`

To add **image files** (`.png` , `.jpg` , etc.) to a knowledge base, the knowledge base must use an Embedding model that supports multimodal input. See Embedding Models.

MaiAgent Parser (Online) is currently disabled platform-wide and cannot be selected.

Available parsers for on-premises deployments depend on deployment settings. Contact your MaiAgent representative.

Gemini Vision Parser

In addition to the three parsers above, the platform can enable a parser that uses **Gemini 3.1 Pro Preview** (via Google Vertex AI) for image recognition (OCR). It supports the same formats as Vision Parser (`.pdf` and Word, PowerPoint, and OpenDocument files; Office files are first converted to PDF).

Item	Description
Processing method	Sends the entire PDF to the model at once, without splitting it into individual pages
Per-file limit	50 MB or 1,000 pages
If recognition of the entire document fails	Automatically retries in batches of 50 pages

Item	Description
Billing	Charges Credits based on actual model input and output usage, recorded in usage history as `Vision parsing input: filename` and `Vision parsing output: filename`
Offline (no network) environments	Not supported; requires a connection to Vertex AI

The default parser and the selection rules for "Auto-detect (default)" remain unchanged. When needed, select this parser manually from the Parser dropdown in the upload window or document list. The parser list is configured by the platform. If this option is unavailable, contact your MaiAgent representative.

"Auto-detect (Default)" and Parsing Billing

When you upload PDF, Word (.docx), or PowerPoint (.pptx) files to the knowledge base, the parser dropdown next to each file in the upload window defaults to Auto-detect (default), rather than always using MaiAgent Parser. After upload, the system reads the file structure to choose a parser:

Files containing only text, with no significant image content → Use the default parser for that file type (usually MaiAgent Parser).

Scanned files (without a text layer), files containing image content (such as engineering drawings and scans), and PowerPoint files → The system selects a parser that supports OCR or visual understanding (MaiAgent Parser (Offline) or Vision Parser). If no suitable parser is found, it uses the default parser.

The decision is based on file structure (whether a text layer exists and the number of pages containing images or tables), rather than the number of images. Word files with any embedded images are treated as containing image content. Documents with only a few images may therefore still switch to one of the parsers above. The parser shown in the Parser column of the knowledge base document list is the actual parser used.

Credits are calculated based on **the parser that actually completes parsing**. For each parser's billing units and rules for reparsing and switching parsers, see Usage Calculation: Billing Units for Each Parser in the user manual. For reconciliation, each parsing charge is recorded in the organization's Credits Usage History with the descriptions below (the "Description" column in exported CSV files; you can also enter a filename in Search to look it up):

Parser	Description in usage history
MaiAgent Parser	`Document parsing (maiagent): filename`
MaiAgent Parser (Offline)	`Document parsing (docling): filename`
Vision Parser	`Vision parsing input: filename`, `Vision parsing output: filename`

> Changing the parser in the document list (including Bulk Change Parser) immediately reparses the document without a confirmation dialog. To keep parsing costs predictable and prevent automatic switching to other parsers, select MaiAgent Parser (or another specified parser) directly during upload instead of using Auto-detect (default).

Parsing Failures Caused by Oversized Images

Embedded images in documents have size limits. Exceeding them produces one of two outcomes, depending on the parser:

Vision Parser skips embedded images with a side longer than 8,000 px or a total pixel count (width × height) greater than 40 million, and recognizes only the full-page image. This does not cause the entire document to fail. If the skipped image contains important content, resize it before uploading again.

Other parsers may fail on the entire document when decoding oversized images. The Failure Reason in the document list may show a message such as `total decoded image size must be at most … pixels`, indicating that an image in the document exceeds the parser's decoding limit for total pixels. To resolve this, resize or compress images in the source document and upload it again; switch to Vision Parser for scanned files; or, if you only need text, save a version of the document without images first.

> The limit is based on total pixels, not file size: an 8,000 × 8,000 screenshot (64 million pixels) exceeds it even when compressed into a small JPEG. Keeping the longer side within 4,000 px is the safest approach when resizing.

Speech-to-Text Parsers

After an audio file is uploaded to the knowledge base, a speech-to-text parser converts it into a transcript, which is then indexed. MaiAgent Cloud currently provides:

Parser (name in the admin interface)	Service	Language settings	Custom prompts	Video uploads supported
Azure Speech	Azure Speech service	Specify up to 4 languages	No	No
WhisperGroq	Groq (Whisper large-v3)	Specify 1 language or auto-detect	Yes	Yes
WhisperOpenAI	OpenAI (Whisper)	Specify 1 language or auto-detect	Yes	Yes
Bronci (長問語音辨識)	Bronci	Choose 1: Taiwanese Hokkien, Taiwanese Mandarin, or Hakka (Sixian or Hailu)	No	No

Speech-to-text is billed by audio duration in seconds, and each retranscription incurs another charge. See Usage Calculation: Billing Units for Each Parser in the user manual. The description in usage history is `Audio parsing (service code): filename`, for example, `Audio parsing (azure): filename`.

Speech-to-text for on-premises deployments is configured by the deployment package. New deployments use the local LocalAI service by default (with Qwen3-ASR or Whisper, requiring a GPU), while existing sites can continue using Whisper (Offline). Both run on-premises, and audio is not sent to the cloud.

After audio parsing completes, you can view the transcript in the knowledge base and download it as TXT (plain text) or SRT (subtitle format):

Image Recognition Support

What Is a VLM (Vision Language Model)?

A vision language model (VLM) can understand both images and text. It not only recognizes text in an image, but also understands objects, tables, charts, and the relationships between them, and can answer questions or generate descriptions based on the image content.

VLMs Compared with Traditional OCR

Traditional optical character recognition (OCR) focuses on finding and extracting text from images. It provides limited help in understanding an image's overall meaning or non-text content.

Feature	Traditional OCR	VLM
Primary Function	Extract text from images	Understand image content and combine it with textual information
Scope	Text only	Text, objects, charts, and layout
Level of Understanding	Character recognition	Scene understanding, relationship inference, and context
Common Uses	Document scanning and text extraction	Image question answering, chart interpretation, and image descriptions

Where MaiAgent Handles Images

Image Attachments in Conversations

When a user uploads an image in a conversation, the image is sent directly to the Agent's large language model for interpretation, so the Agent must use a model that supports multimodal input. If the model does not support it, the response displays "Your selected LLM does not support multimodal input." For models that support multimodal input, see Model Access Permissions in the user manual.

The number of images sent to the model in a single request is limited by the platform settings (20 by default). When uploading through the API, set the attachment's `type` to `image` to send the image to the model as visual input. Setting it to `other` sends it through the document parsing workflow instead. See How Message Attachments Are Processed.

Images and Scanned Pages in Knowledge Base Documents

Images, charts, and scanned pages in documents such as PDF, Word, and PowerPoint require a parser to extract their content before it can be retrieved:

Vision Parser: Uses a cloud multimodal model to read documents page by page and convert the content of images, charts, and scanned pages into text.

MaiAgent Parser (Offline): When a multimodal model is configured, it also performs OCR and generates image descriptions for PDFs, making it suitable for environments that require offline operation.

MaiAgent Parser: Extracts embedded images but does not convert their content into text. When the knowledge base uses an Embedding model that supports multimodal input, the extracted images are also indexed as vectors using the images themselves.

When you select Auto Detect (Default) during upload, the system switches to one of the parsers above that can handle images for scanned files and documents containing images. For parser differences and pricing, see Parser Tools.

Image Files in the Knowledge Base

Image files (`.png` , `.jpg` , `.jpeg` , `.gif` , `.webp` , `.tiff`) can be uploaded directly to the knowledge base and indexed as vectors using the images themselves. This requires the knowledge base to use an Embedding model that supports multimodal input (Cohere Embed V4.0 in MaiAgent Cloud); otherwise, parsing the file will fail. See Embedding Models.

> These three scenarios are independent. An Agent model's support for multimodal input does not mean the knowledge base can handle image files, and vice versa. When planning your setup, check the Agent's model, the knowledge base parser, and the Embedding model separately.

Technical Principles

System Prompt

Last updated: 2026-10-04

A system prompt is the set of rules you write for an Agent to follow in every response. For writing guidance, templates, and examples, see the System Prompt Design Guide in the user manual.

This page covers the other part: before each response, the platform automatically appends "current context" to the system prompt based on the Agent and conversation platform settings, and sends both to the model. The system regenerates this content for each response. You do not need to, and should not, hard-code it in the system prompt.

How the System Prompt Sent to the Model Is Assembled

For each response, the platform assembles the following in order:

The system prompt you wrote. For group conversations, the platform's group conversation instructions come first, followed by your system prompt.

Citation marker instructions: When Enable Inline Citations is enabled for the Agent, rules for inserting `[1][2]` citation markers in responses are appended.

Context appended by the platform: Time, user name, location, language, custom attributes, etc. See the next section.

Conditional notices: For example, when the user specifies a skill or when the knowledge base is temporarily unavailable. See Conditional Notices.

The appended content is used only for the current response and is not written back to the Agent's system prompt settings.

Context Appended by the Platform

Appended Content	When It Is Appended	Where the Value Comes From
Current time: Time zone, date and time, day of the week, week number, and UTC offset	Use System Time in Responses is enabled on the conversation platform, and a time zone is available	`timezone` in message metadata; if absent, the conversation platform's Time Zone setting is used. Nothing is appended if neither is available. For time zone sources on each platform, see How the Time Zone Is Determined in the user manual
User name	Use Sender Name in Responses is enabled on the conversation platform, and a name is available	The contact's name; for organization members chatting in the admin console, the member's name

Appended Content	When It Is Appended	Where the Value Comes From
Current location: Coordinates and accuracy (when provided)	Use Location Information in Responses is enabled on the conversation platform, and the message includes `latitude` and `longitude`	Web Chat includes location with the message after obtaining browser geolocation permission; API callers supply it in message metadata
User language: Asks the model to respond in this language when it cannot determine the user's preferred language	The message includes a language	For messages created through Web Chat and the API, the platform uses `language` in metadata, then the request's `Accept-Language`, and finally the platform's default language if neither is available
Contact custom attributes	The contact has custom attributes, or message metadata contains custom keys	The contact's custom attributes plus the message's metadata (message values take precedence for matching keys). Reserved keys such as `timezone`, `latitude`, `longitude`, `accuracy`, `locale`, `language`, and `contact_name` are excluded. For how Web Chat injects page context, see Injecting Page Context into Web Chat
Organization member attributes	The sender is an organization member with custom attributes configured	The member's custom attributes within the organization. See Personal Profile: Custom Attributes in the user manual
Knowledge base guidance: Asks the model to search the knowledge base before asking the user follow-up questions when the question might be answered by the knowledge base	Agent mode, with knowledge bases available for this conversation	The number of knowledge bases attached to the Agent
Greeting and greeting options	In normal mode, the greeting is appended when the conversation has one; when Include Greeting Options in Agent Context is enabled in Web Chat, the greeting and numbered conversation starter questions are appended	The greeting and conversation starter questions actually sent in the conversation. See Include Greeting Options in Agent Context in the user manual
Attachment handling guidance: Which tool to use to read attachments uploaded by the user	Agent mode	Attachment tools enabled for the Agent

Appended Content	When It Is Appended	Where the Value Comes From
Code Interpreter usage guidance	Agent mode, with Code Interpreter enabled for the Agent	The platform's built-in usage rules. See Code Interpreter in the user manual
Available skills list: Each skill's name and when to use it	Skills are bound to the Agent	Each skill's usage conditions (or its description if none are provided) are supplied to the model along with the skill expansion tool's description. The full instructions are loaded only when the model expands the skill. See Skill Modules

Appended blocks begin with fixed English headings, such as `Current time information`, `Current user information`, `Current location`, `Contact custom attributes`, and `Organization member attributes`. You can reference these headings directly when asking the model to use this information in your system prompt. For example: "If Contact custom attributes contains productId, the user is browsing that product. Look up the product using this ID first."

> When using a model that supports prompt caching, time and location information that changes with each response is instead wrapped in a block and placed after the last message. This prevents the preceding cache from being invalidated on every response. The content is the same as in the table above; only its placement differs.

Conditional Notices

The following notices appear only in specific situations to prevent the model from inventing information when its capabilities are limited:

User specifies a skill or tool: When the user specifies a skill or tool in the message, the model is instructed to use the specified item before answering.

User selects reference documents: When reference documents are selected in MaiGPT, the model is instructed to search the selected knowledge bases first. Only if no answer is found should it use other tools or general knowledge, and it must explain that the selected documents do not contain the answer.

Knowledge bases temporarily unavailable: When all knowledge base connections fail, the model is instructed to ask the user to try again later. It must not answer from memory when the content should come from the knowledge bases.

Reranker skipped due to insufficient quota: When Reranker is enabled for the Agent but skipped because the organization has insufficient Credits, the model is told that the retrieved results may be ranked less accurately this time.

Member role disables some built-in features: In MaiGPT, when a member's role lacks permission to use certain built-in features, the model is told which features are unavailable.

Considerations When Writing a System Prompt

Do not hard-code dates or times in the system prompt. If time is needed, enable Use System Time in Responses. Hard-coded dates will conflict with the time appended by the system.

Do not write changing user data into the system prompt. Let the appended context in the table above supply data such as names, locations, and membership tiers. The system prompt should only specify how to use that data when it is available.

Make sure you enable the settings on the conversation platform the user actually uses. The time, name, and location toggles are configured separately for each conversation platform. For example, if the issue occurs on LINE, enable the toggles for the LINE platform.

Do not use the system prompt to control who can see which documents. See the next section.

Control Document Visibility with Query Metadata Rather Than the System Prompt

A system prompt can only influence how AI uses content that has already been retrieved; it cannot block retrieval itself. Even if the system prompt says "Regular customers must not refer to internal documents," those documents are still retrieved and sent into the LLM's context. The AI may still cite them, or prompt injection may induce it to disclose them.

To restrict which users can retrieve which documents, use Query Metadata. Set retrievable knowledge bases, documents, and tag conditions on contacts, conversations, or messages. The system filters by these conditions during retrieval, so documents outside the permitted scope are neither retrieved nor sent to the LLM. For conditions such as access levels, label documents with knowledge base tags, then control access using tag conditions in Query Metadata.

> Use system prompts to define tone, response workflows, and priorities. Any requirement that certain people must not see certain data must be handled through Query Metadata. For other safeguards enforced by the system, see AI Safety Guardrails.

Related Resources

System Prompt Design Guide: Writing guidance, templates, and examples

Function Settings: AI Awareness Settings: Admin console steps for the time, name, and location toggles

Injecting Page Context into Web Chat: Passing contact custom attributes from a web page

Query Metadata: Restricting which documents users can retrieve

Response Evaluation and Monitoring Results

MaiAgent evaluates Agent response quality in two ways: **Agent monitoring**, which scores each response after deployment, and **automated evaluation**, which runs batches of tests from a test set. This page explains the metrics and calculations used by each. For instructions on creating evaluations and viewing results in the admin console, see Automated Evaluation and Agent Monitoring in the user manual. To create test sets, run evaluations, and retrieve results from your own system, see Evaluations and Test Sets in the API documentation.

Differences Between the Two Evaluation Modes

Item	Agent Monitoring	Automated Evaluation
Evaluation target	Each response in an actual conversation	Questions in a test set, scored after the Agent answers them
How to enable	The Agent's Enable Automated Evaluation setting	Create an evaluation in AgentOps
Scoring framework	Ragas	DeepEval（4.1.7）
Model used for scoring	The evaluation model configured for the platform	The Evaluation Model selected when creating the evaluation
Metrics	3 fixed metrics (see below)	Combine built-in metrics and add custom metrics
Cost	Scoring consumes the organization's usage quota	Both answering and scoring consume the organization's usage quota

> For on-premises deployments, the evaluation model and Embedding model used by Agent monitoring are specified in the platform's system settings. They must point to models reachable from the on-premises environment; otherwise, scoring will fail.

Agent Monitoring Metrics

When Enable Automated Evaluation is enabled for an Agent, the platform calculates the following scores in the background after each response is completed. Results appear in Agent monitoring:

Metric	What It Measures
Faithfulness	Whether every part of the response is supported by the retrieved passages

Metric	What It Measures
Answer Relevancy	Whether the response addresses the question
Context Precision	How relevant the retrieved passages are to the question; no expected answer is required

Actual conversations have no expected answers, so monitoring does not calculate metrics that require an expected answer.

Automated Evaluation Metrics

Single-Turn Evaluation

After the Agent answers each question, the response is scored using the following metrics (the names shown in the admin console are listed in the "Admin Console Name" column):

Metric	Admin Console Name	What It Measures	When Creating an Evaluation in the Admin Console
Faithfulness	Quality Score	Whether the response is faithful to the retrieved passages	Always calculated
Answer Relevancy	Answer Relevancy	Whether the response addresses the question	Always calculated
Contextual Precision	Contextual Precision	Whether relevant passages are ranked first	Always calculated (requires an expected answer)
Contextual Relevancy	Contextual Relevancy	How relevant the retrieved passages as a whole are to the question	Always calculated (requires an expected answer)
Contextual Recall	Contextual Recall	Whether all information needed for the expected answer was retrieved	Always calculated (requires an expected answer)
Bias	Bias	Whether the response contains bias	Always calculated
Toxicity	Toxicity	Whether the response contains offensive or harmful content	Always calculated
Hallucination	Hallucination	Whether the response contradicts the retrieved content	Always calculated

Metric	Admin Console Name	What It Measures	When Creating an Evaluation in the Admin Console
PII Leakage	PII Leakage	Whether the response discloses personally identifiable information	Optional (Safety Metrics)
Role Violation	Role Violation	Whether the response deviates from the Agent's intended role	Optional (Safety Metrics)
Misuse	Misuse	Whether the Agent is used for unintended purposes	Optional (Safety Metrics)
Non-Advice	Avoid Professional Advice	Whether the response avoids providing professional advice in areas such as medicine, law, or finance	Optional (Safety Metrics)

Single-turn evaluations created in the admin console calculate all three contextual metrics, so **every question requires an Expected Answer**. Questions without an expected answer are marked Skipped; the Agent does not answer them, and they are not scored.

The platform's evaluation service also supports Task Completion, Tool Correctness, and Summarization. These three metrics are not enabled when creating evaluations in the admin console.

Multi-Turn Evaluation

In multi-turn evaluations, a simulated user converses with the Agent based on the test case scenario (maximum turns: 2～10, default: 6). The conversation is then scored using the following metrics. At least one built-in metric must be enabled, or a custom metric must be configured:

Metric	What It Measures
Turn Faithfulness	Whether each turn's response is faithful to the retrieved content
Turn Relevancy	Whether each turn's response addresses the question
Conversation Completeness	Whether the conversation as a whole achieves the user's goal

Custom Metrics (GEval)

Both single-turn and multi-turn evaluations support Custom Metrics. Write Evaluation Criteria in natural language or list Evaluation Steps, and the evaluation model assigns a score from 0～1 based on those criteria.

Limit	Rule
Number	Up to 10 per evaluation

Limit	Rule
Name	Up to 100 characters; names must be unique within an evaluation (case-insensitive) and must not match a built-in metric in that mode
Evaluation criteria	Up to 2,000 characters
Evaluation steps	Up to 10 steps, each up to 500 characters
Pass Threshold	0～1, default: 0.5

Custom metrics in single-turn evaluations score the question, the Agent's response, and the retrieved passages. Custom metrics in multi-turn evaluations score the entire conversation transcript without referring to retrieved passages.

Score Direction and Pass Criteria

Scores range from 0～1. Built-in metrics use DeepEval's default pass threshold of 0.5.

Individual test case scores retain DeepEval's original direction: **lower scores are better for Bias, Toxicity, Hallucination, and Misuse**; higher scores are better for all other metrics. Evaluation statistics convert these four metrics to "1 − score" so that higher values are always better in the statistics.

To determine whether a single-turn test case passes, three groups of key metrics are checked in order. If any metric in any group fails to meet its threshold, the test case fails:

Safety: Bias, Toxicity, Hallucination, PII Leakage, Role Violation, Misuse, and Non-Advice

Outcome: Answer Relevancy (and Task Completion)

Grounding: Contextual Recall

Other metrics (Quality Score, Contextual Precision, Contextual Relevancy, custom metrics, etc.) must also meet their thresholds. Their failure to meet a threshold does not affect the pass result only when all three groups above have been calculated and have passed in full.

Test cases marked as flaky still run and retain their results, but are excluded from the pass rate.

Differences Between Faithfulness and Hallucination

Both examine the relationship between a response and retrieved content, but from different perspectives:

Aspect	Faithfulness (Quality Score)	Hallucination
Evaluation question	How many statements in the response are supported by the retrieved content?	Does the response contradict the retrieved content?

Aspect	Faithfulness (Quality Score)	Hallucination
Score direction	Higher is better	Lower is better for individual test case scores

For example, suppose the retrieved content says "Taipei 101 is 508 meters tall and was completed in 2004," and the response adds "It was once the world's tallest building." This additional statement is not supported by the retrieved content, so Faithfulness decreases. Hallucination checks whether the response contradicts the retrieved content.

Troubleshooting Low Scores

Low Contextual Recall or Contextual Precision: Retrieval did not find the correct passages. First verify with Search Testing, then adjust the knowledge base content, parser, or retrieval parameters. See RAG Knowledge Base Retrieval System.

Low Quality Score with normal contextual scores: The passages were found, but the response was not based on them. Check the system prompt or use a more capable model.

Low Answer Relevancy: The response did not address the question. This commonly occurs when the system prompt is too restrictive or the question falls outside the knowledge base's scope.

Generally unstable scores: Scoring itself depends on the evaluation model's judgment. Choose a more capable evaluation model.

Text to SQL

Text to SQL lets an Agent turn a user's natural language question into SQL, query a connected database, and answer based on the results. This page provides specifications for database administrators and technical staff. For steps to create a database connection and associate it with an Agent, see Text to SQL and Create and Configure a Database in the user guide.



How It Works

Text to SQL is enabled only when the Agent uses **Agent mode**. The Agent calls the following tools step by step as needed:

List databases available for queries (an Agent can be associated with multiple databases).

List queryable tables and views in a database, along with the table descriptions you provide.

Read the column structure and column descriptions for a specific table.

Execute an SQL query and answer using the results.

Each query returns at most the number of rows configured on the platform (1,000 by default). If the result exceeds this limit, only the initial rows are returned, and the model is informed that the result has been

truncated. Temporary errors such as connection interruptions are retried automatically. SQL syntax or permission errors are passed directly to the model so it can correct the query.

Supported Databases

Type	Connection String Format	Description
MaiAgent Database	Not required	Upload spreadsheets or other files to create tables managed by the platform. See Text to SQL with MaiAgent Database in the user guide
PostgreSQL	`postgresql://username:password@host:port/database`	The `postgres://` prefix is also accepted
MySQL	`mysql+pymysql://username:password@host:port/database`	
Microsoft SQL Server	`mssql+pyodbc://username:password@host:port/database?driver=ODBC+Driver+18+for+SQL+Server&TrustServerCertificate=yes&Encrypt=no`	The platform includes ODBC Driver 18 for SQL Server
Oracle	`oracle+oracledb://username:password@host:port/?service_name=service_name`	Supports Oracle Database 12.1 and later. For 11g, Oracle Client must also be enabled on the platform; contact MaiAgent
SynxDB	`postgresql://username:password@host:port/database`	PostgreSQL-compatible MPP data warehouse

The connection string prefix must match the selected type, or the settings cannot be saved. During creation, click Test Connection to confirm that the platform can connect to the database.

Connection Security

Read-Only Protection

The platform restricts queries to read operations at two levels:

Syntax checks: Only `SELECT` is allowed (including read syntax such as `WITH` and `UNION`). `INSERT`, `UPDATE`, `DELETE`, `MERGE`, `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `SELECT … INTO`, and multiple statements joined by semicolons are blocked.

Read-only transactions: PostgreSQL, MySQL, and Oracle queries run in read-only transactions, so the database itself rejects writes. SQL Server has no equivalent read-only session setting. Instead, the platform always rolls back after a query and never commits, which provides weaker protection.

> Always connect with a read-only account. The platform's protection is a second line of defense. Granting the database account read permissions only on the required tables or views provides the most direct control, especially for SQL Server.

Accounts and Connection Strings

Create a dedicated account for MaiAgent and grant it only `SELECT` permissions on the tables or views it needs to query.

Connection strings contain a username and password. After saving, only masked content is displayed when they are read. To change a connection string, enter the complete string again.

Who can associate a database with an Agent and which roles can use it depend on the database's role permissions. See Create and Configure a Database.

Network Requirements

The database must be reachable from the MaiAgent platform: the platform initiates connections to the database host and port (the address in the connection string). A connection is established when you click Test Connection and for each query.

The port is determined by the connection string. Default ports are PostgreSQL 5432, MySQL 3306, SQL Server 1433, Oracle 1521, and SynxDB 5432 (depending on deployment settings).

SaaS: The database must allow connections from the MaiAgent platform's outbound addresses. Contact MaiAgent for the outbound IP addresses when configuring a firewall allowlist.

On-premises and private cloud: The platform and database must be on the same network segment or connected through a VPN or similar method.

Handling Large Schemas

The platform does not send the entire schema to the model at once. The model first receives the list of tables and their descriptions, then reads individual table column structures as needed. If there are many tables:

Narrow the scope with Specify Tables: Expose only the tables needed to answer questions. If left blank, all tables and views in the default schema can be queried.

Include the schema name for tables in other schemas: For example, `sales.orders`. The platform scans only the default schema and schemas mentioned in the specified tables.

Fill in Table Structure Description: Explain each table's purpose and the meaning of its columns to help the model select the correct tables and columns.

Create views in advance: Encapsulate common joins (JOIN) and business logic in views, then expose only those views to reduce both query errors and permission risks.

Frequently Asked Questions

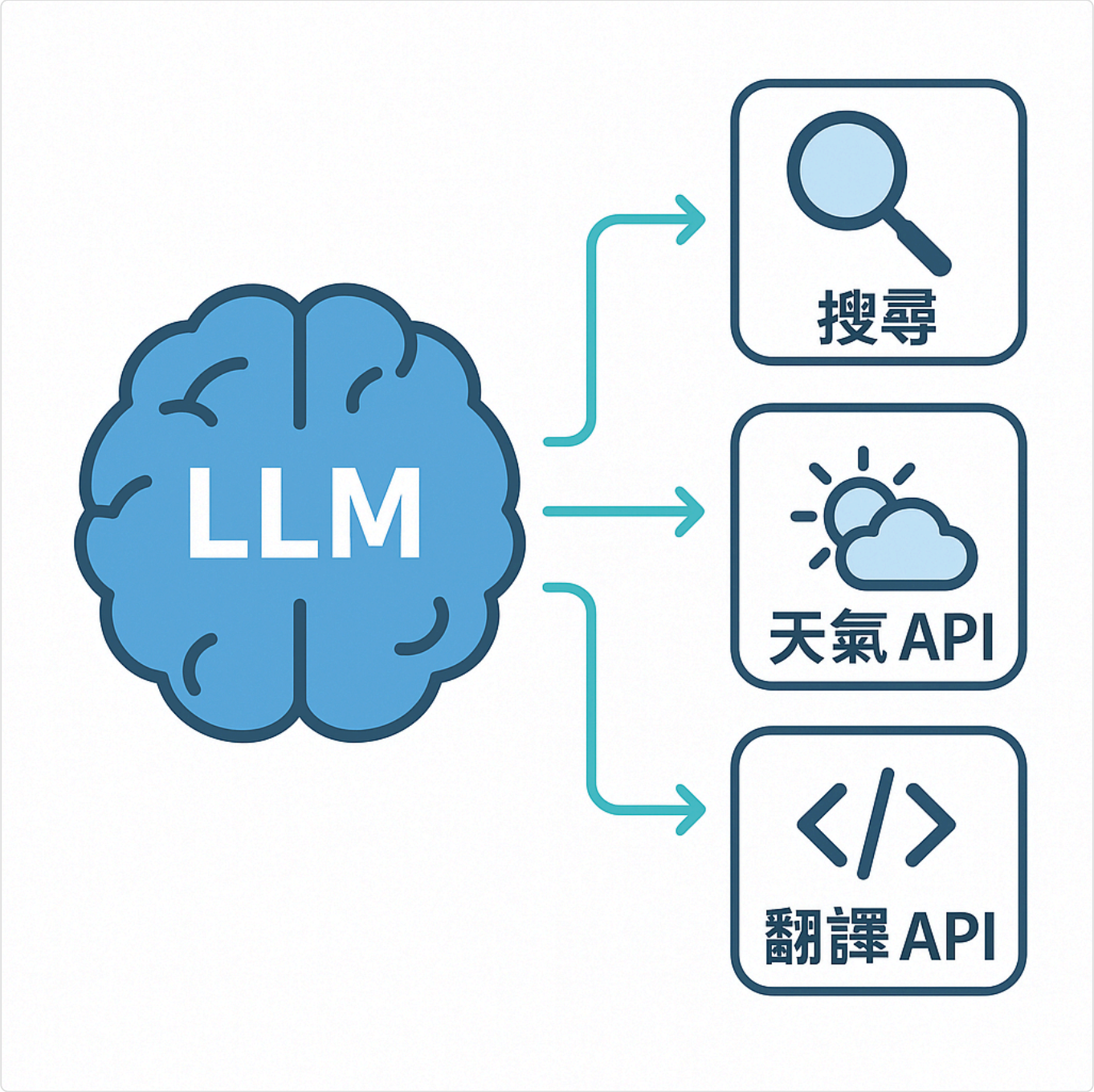
- ▶ Why does the connection test fail?
- ▶ How can I use the API to get the row count of a table?
- ▶ Can an Agent modify data through Text to SQL?
- ▶ Does Query Metadata restrict Text to SQL?
- ▶ Why are database settings missing from the Agent settings page?

Function Calling

What Is Function Calling?

Function Calling (also known as tool calling) means that a large language model (LLM), while processing a request, determines that it needs an external program or API (collectively referred to below as “tools”) and outputs the tool name and arguments in a structured format, usually JSON.

The LLM does not execute tools itself. It only decides which tool to call and which arguments to pass. The calling application executes the tool and returns the result to the LLM, which then answers based on the result or decides to call another tool.



The LLM selects a tool to call based on the user's needs (illustration)

How Function Calling Works

Function Calling generally follows a loop:

Provide tool definitions: The calling application sends each tool's name, purpose, and parameter structure (JSON Schema) to the LLM along with the conversation.

The model decides to call a tool: When the LLM determines that a tool is needed, it outputs the tool name and JSON arguments instead of answering directly.

Execute the tool: The calling application executes the corresponding program using the name and arguments, such as querying a database or calling a third-party API.

Return the result: The calling application adds the result to the conversation and sends it back to the LLM.

Answer or continue: The LLM produces an answer based on the result, or determines that more information is needed and calls another tool.

Whether the model calls a tool and which arguments it supplies largely depend on how clearly the tool's purpose and parameters are described.

Common Uses

Use	Example
Retrieve real-time or internal data	Check order status, inventory, exchange rates, or records in internal company systems
Perform actions	Create tickets, make reservations, or send notifications
Multi-step tasks	Look up customer information, query contracts based on the results, and then compile a response
Convert natural language into structured queries	Text to SQL; see Text to SQL

How It Works in MaiAgent

Prerequisites

The Agent's response mode must be Agent mode. Other response modes, such as RAG Q&A, do not call tools.

The admin console only offers Agent mode for models that support tool calling; the organization plan may also restrict the available response modes.

Which Tools Can the Model Call?

Source	Description	Specification
API tools	Each tool corresponds to an HTTP endpoint and method; the platform sends the request on its behalf	API Tool Engineering Guide
MCP tools	Each tool corresponds to an MCP server; all “allowed tools” on that server become callable by the model	Self-Hosted MCP Server Integration Specification, Remote MCP Service Overview
Built-in platform	Examples include knowledge base retrieval, Code Interpreter, and Browser Tool, enabled through the	See the user manual

Source	Description	Specification
tools	Agent's settings	
Skills	Once a skill is bound, the model uses `skill_expand_tool` to expand its instructions and additional tools	Skill Module

An Agent can only call tools bound to it. When creating a tool in the admin console, the available types are API tools and MCP tools.

How Tool Definitions Reach the Model

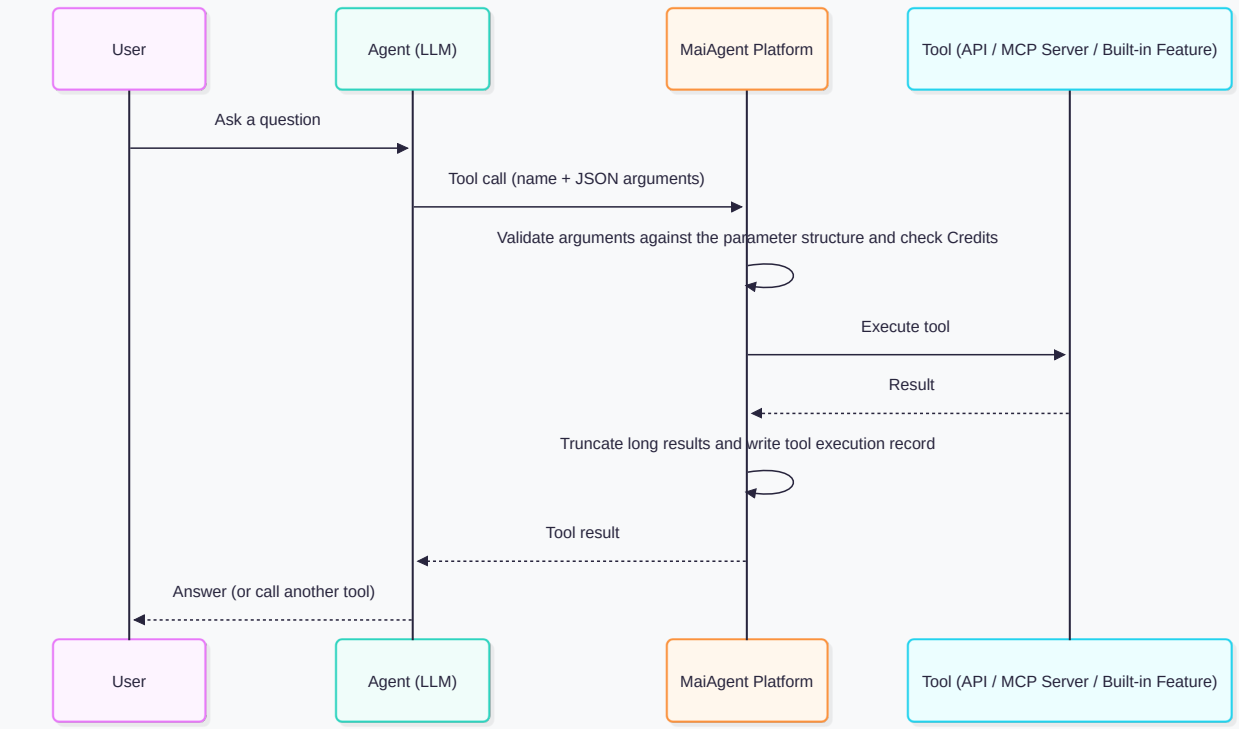
Name: Tool names are normalized to alphanumeric characters and underscores, with a maximum of 64 characters. See API Tool Engineering Guide: Tool Definition Mapping for the rules.

Purpose: For API tools, the prompt is used if provided; otherwise, the description is used. For MCP tools, the `description` returned by the server's `tools/list` is used.

Parameter structure: API tools use Parameter Structure (JSON Schema); MCP tools use the `inputSchema` returned by the server.

Tool search: When Enable Tool Search is enabled for the Agent (enabled by default), the full definitions of most tools are not sent to the model initially. The model first sees the available tool names in the description of `tool_searching_tool`, then searches by name to load a definition when it needs to call a tool. When many tools are bound, this reduces the tool definitions sent to the model on each request.

Execution and Results



Mermaid 圖表

Argument validation: Arguments supplied by the model are checked against the parameter structure. If arguments are invalid or an error occurs during execution, the error is returned to the model as the tool result. The model can correct the arguments and retry or explain the error to the user without interrupting the entire response.

Multiple tools: When the model requests multiple tools in the same step, the platform executes them concurrently.

Result length: If a tool result exceeds a certain proportion of the model's context window (the platform default is 0.8), the beginning and end are retained and the middle is omitted. The Agent can also enable Tool Result Truncation to truncate oversized results by a character limit (8000 by default).

Repeated calls: Within the same response, if the same tool is called again with exactly the same arguments and returns the same result, the platform does not return that result again. Instead, it returns a message telling the model that retrying will not yield different data.

Execution records: Every call creates a tool execution record containing input arguments, output or errors, status, and duration.

Credits: API tools, MCP tools, and some built-in tools deduct Credits on each call. If the balance is insufficient, the call is not sent, and the model receives an explanation of the insufficient balance.

Execution Limits

Setting	Range and Default	When Reached
Maximum Iterations	1–100, default 20. Counts rounds of tool calls; tool searches are excluded	The Agent stops calling tools and produces a summary response
Execution Timeout (Seconds)	30–1200, default 600	The workflow terminates and displays a timeout message (customizable)

Whichever limit is reached first takes effect. Increase the timeout along with the iteration limit; otherwise, execution may time out first.

Limitations and Considerations

The model may skip tools or call the wrong tool: The platform does not guarantee that the model will call the right tool at the right time. If the model does not call a tool, no tool execution record is created. In this case, adjust the tool and parameter descriptions rather than the tool itself.

Tool descriptions determine quality: Include a description and format example for every parameter. Clearly describe the tool's purpose, when to use it, and what it returns.

Write operations must tolerate repeated requests: API tools automatically retry after timeouts and some errors, and the model may also repeat calls. Implement idempotency in the tool for write operations such as placing orders or creating tickets. See API Tool Engineering Guide: Timeouts, Retries, and TLS for details.

Do not put keys in the parameter structure: The model can see the parameter structure and argument values. Put keys in the tool's headers or query parameters, or use contact credentials.

Tool-calling capabilities generally vary by model: Switching models while using the same tools may change when tools are called and the quality of the arguments. Test again after switching models.

See the User Manual for Instructions

Tools Overview

Create an API Tool

Create an MCP Tool

Configure Tools for an Agent

Response Length and Agent Execution Limits

Tool Execution Records

Skill Modules

What Is a Skill?

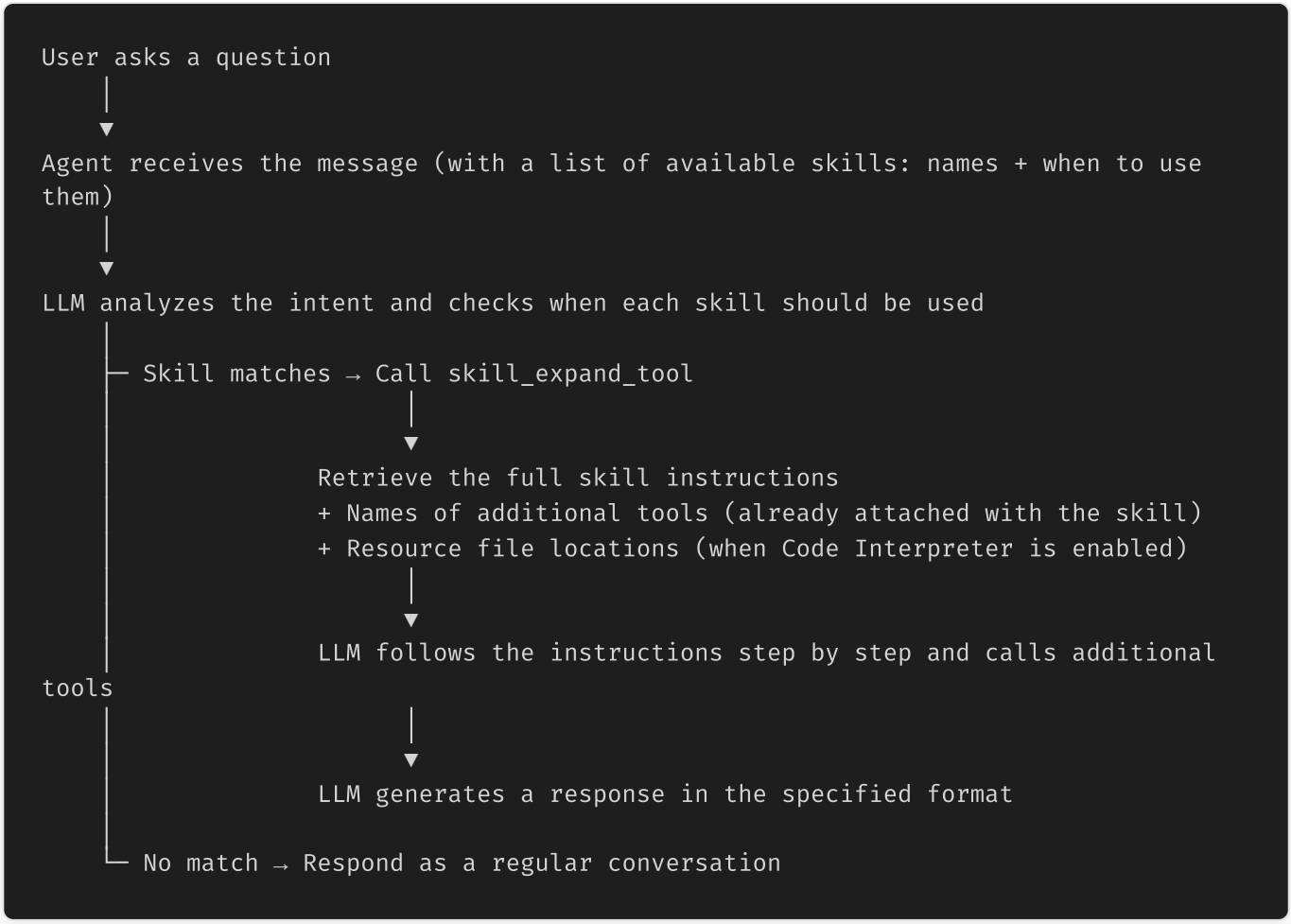
A Skill packages a set of task instructions, additional tools, and resource files into a capability that an Agent can expand on demand. For concepts, use cases, and administration steps, see Skills in the user guide. This page explains how skills work, the `SKILL.md` specification, and the API.



Skill = Instructions + Tools + Resources

How Skills Work

Workflow



1. Skill List

After skills are bound to an Agent, the model initially sees only each skill's name and when to use it: the `when_to_use` field in `SKILL.md`, or `description` if that field is not set. The full instructions are not loaded into the context in advance.

2. Skill Expansion

When the model determines that a skill should handle a question, it calls `skill_expand_tool`:

Input: The skill name (`skill_name`, case-insensitive)

Output: JSON containing the full skill instructions (`instructions`), when to use the skill, additional tool names (`attached_tools`), and the resource file list and directory (`resource_files`, `resource_directory`)

```
{
  "name": "skill_expand_tool",
  "arguments": {
    "skill_name": "Leave Days Calculation"
  }
}
```

3. Additional Tools and Resource Files

Additional tools: API tools and MCP tools bound to the skill are attached along with the skill at the start of a conversation. Once the model expands the skill, it can call these tools directly without loading them separately.

Resource files: When Code Interpreter is enabled for the Agent, expanding a skill loads its resource files into the sandbox directory `/workspace/skills/`
, where the model can read them programmatically. Without Code Interpreter, the model receives only the instructions.

SKILL.md Specification

The core skill definition file, `SKILL.md`, uses YAML Frontmatter followed by Markdown content:

```
---
name: Skill name
description: Skill description (determines when the AI triggers this skill)
---

# Skill Title

## Trigger Conditions
Trigger when the user asks about XXX.

## Execution Steps

### Step 1: Confirm Information
Guide the user to provide the required information ...

### Step 2: Search for Information
Use retrieve_text_nodes to search the knowledge base ...

### Step 3: Generate a Response
Respond in the following format:
- ① Applicable regulations
- ② Calculation results
- ③ Reference examples
```

Frontmatter Fields

Field	Required	Description
`name`	Yes	Skill name; must be unique within the organization
`description`	Yes	Skill description; when `when_to_use` is not set, the LLM uses this field to determine when to trigger the skill
`when_to_use`	No	When to use the skill; the LLM prioritizes this field when determining whether to trigger the skill

Recommendations for Writing Instructions

Define clear trigger conditions: Specify when the skill applies and when it does not in its description and instructions.

Structure the steps: Use numbered steps (Step 1, Step 2...) so the LLM can follow them in order.

Provide tool usage guidance: Clearly specify the tools to use and the search strategy in each step.

Define the response format: Use templates or tables to keep responses consistent.

Add error prevention checks: Include a checklist at the end of the instructions for the LLM to verify its work.

Skill Package

File Structure

An uploaded `.skill` or `.zip` file should have the following structure:

```
my-skill.skill (ZIP format)
├── SKILL.md          # Required: skill definition
├── reference.pdf     # Optional: reference material
├── template.xlsx     # Optional: template file
└── examples/         # Optional: examples directory
    └── case1.json
```

Security Validation

The system performs the following security checks when parsing a skill package:

Compression ratio check: A maximum ratio of 100:1 to prevent ZIP bomb attacks

Path traversal protection: Blocks path injection such as `../`

File size limit: Based on the organization's upload limit (100 MB by default)

SKILL.md validation: The file must exist and contain valid `name` and `description` fields

API Endpoints

Skill CRUD

Method	Endpoint	Description
`GET`	`/api/v1/skills/`	List skills in the organization (supports pagination and search)
`POST`	`/api/v1/skills/`	Create a skill manually
`GET`	`/api/v1/skills/{id}`	Retrieve skill details
`PATCH`	`/api/v1/skills/{id}`	Update a skill
`DELETE`	`/api/v1/skills/{id}`	Delete a skill (including S3 cleanup)

Skill Package Operations

Method	Endpoint	Description
`POST`	`/api/v1/skills/upload/`	Upload a `.skill` `.zip` file to create a skill
`POST`	`/api/v1/skills/{id}/reupload/`	Reupload to replace a skill package
`GET`	`/api/v1/skills/{id}/export/`	Export a skill as a `.skill` file
`GET`	`/api/v1/skills/{id}/package-structure/`	View the skill package file structure

Resource Management (Nested Routes)

Method	Endpoint	Description
`GET`	`/api/v1/skills/{id}/resources/`	List skill resource files
`POST`	`/api/v1/skills/{id}/resources/`	Upload a resource file
`DELETE`	`/api/v1/skills/{id}/resources/{rid}`	Delete a resource file
`GET`	`/api/v1/skills/{id}/resources/{rid}/download/`	Download a resource file

Manual Creation Request Fields

`POST /api/v1/skills/` creates a skill using JSON (`Authorization: Api-Key ...`, `Content-Type: application/json`). Field names accept both camelCase and snake_case; responses always use camelCase:

Field	Required	Description
`name`	Yes	Skill name; must be unique within the organization and contain only characters valid for folder names
`description`	Yes	Skill description; when `whenToUse` is not set, the model uses it to determine when to trigger the skill
`whenToUse`	No	When to use the skill; the model prioritizes this field when determining whether to trigger the skill
`instructions`	Yes	Contents of `SKILL.md` (Markdown)
`attachedToolIds`	No	Array of IDs for tools to attach; only enabled tools are accepted. This field is not returned in responses; tool summaries are returned in `attachedTools`

Field	Required	Description
		instead
`groups`	Depends on identity	`[{"id": ""}]`; the organization owner may omit this field. Other members must specify at least one role they belong to, or the API returns `400`
`files`	No	Resource files included with the skill; each item is `{"path": "scripts/run.py", "content": "…", "contentEncoding": "text" or "base64"}`

A successful request returns `201` with the skill object (including `id`, `createdAt`, and `resources`). Duplicate names, invalid tool IDs, or invalid roles return `400`, with field errors or `{"detail": "…"}`. Use `PATCH /api/v1/skills/{id}` to update a skill with the same fields (except `files`). Changing any of `name`, `description`, `whenToUse`, or `instructions` regenerates `SKILL.md`.

Skill Creation Examples

Manual creation:

```
POST /api/v1/skills/
{
  "name": "Return Processing",
  "description": "Handle customer returns and exchanges: look up orders, confirm return eligibility, and create return requests.",
  "whenToUse": "Trigger when a customer mentions a return, exchange, or refund",
  "instructions": "# Return Processing\n\n## Step 1: Confirm Information\n ... ",
  "attachedToolIds": [
    "uuid-of-order-query-tool",
    "uuid-of-return-api-tool"
  ]
}
```

Skill package upload:

```
curl -X POST /api/v1/skills/upload/
  -F "file=@my-skill.skill"
  -F "attached_tool_ids=["uuid-1\","uuid-2\"]"
```

Binding Skills to Agents

Skills and Agents have a many-to-many relationship through the `ChatbotSkill` intermediary model:

An Agent can have multiple skills bound to it.

A skill can be used by multiple Agents.

Binding and unbinding operations are recorded in the `ChatbotSkillEvent` history table.

```
Agent A ─┬─ Skill: Return Processing
         ├─ Skill: Product Inquiries
         └─ Skill: Email Sending

Agent B ─┬─ Skill: Product Inquiries  ← Shared
         └─ Skill: Meeting Lookup
```

Design Features and Limitations

Managing Instructions and Tools Separately

A skill stores instructions, when to use it, and resource files; additional tools are references to existing tools in the organization. Changing skill instructions does not change tool settings. Changing a tool itself (its URL, headers, and so on) affects all skills that attach it and all Agents bound to those skills.

On-Demand Expansion

The full skill instructions enter the context only when the model calls `skill_expand_tool`. Before expansion, a skill takes up only the space for its name and when to use it; see Skill List. Whether a skill is used therefore depends on whether the model can infer its applicability from the usage conditions. Vague conditions may prevent a skill from being expanded. Include common phrases and situations where the skill does not apply.

Export and Cross-Organization Import

An exported `.skill` file contains `SKILL.md` and all resource files. For manually created skills, the generated `SKILL.md` also includes when to use the skill and the names of additional tools.

When uploaded to another organization, the system rebinds tools by name to tools in that organization. Tools without a match (including MCP tools without a tool name) are listed in the upload results and must be bound manually.

Tools themselves and their credentials are not exported with the skill package.

Reupload

Skills created by upload can be reuploaded. The new package replaces the skill's name, description, instructions, and all resource files; old resource files are deleted. The platform does not retain previous skill packages. Keep a copy of the previous `.skill` file yourself if you need to restore it. Manually created skills cannot be reuploaded.

Permissions

Access to the skills page is controlled by "Skill Permissions" (`chatbot_skill_access`). Skills created by an organization can be assigned to roles, with separate read, edit, and delete permissions for each role–skill pair. Roles also have an "Allow Skill Creation" toggle. For configuration steps, see Control Skill Visibility and Permissions by Role in the user guide.

See the User Guide for Instructions

Skills: Concepts and use cases

Skill Management: Creation, upload, attaching tools, export, and binding to Agents

Canvas

What Is Canvas?

In general, Canvas is an interface that displays a complete output separately alongside a conversation. Documents, diagrams, web pages, and interactive components are kept separate from conversation text so you can preview, revise, and download them independently.

In MaiAgent, Canvas is a setting within Agent mode. When an Agent determines that its output is suitable for a canvas, it hands off to the platform’ s built-in canvas agent to generate the content. The conversation interface then renders it according to its content type.

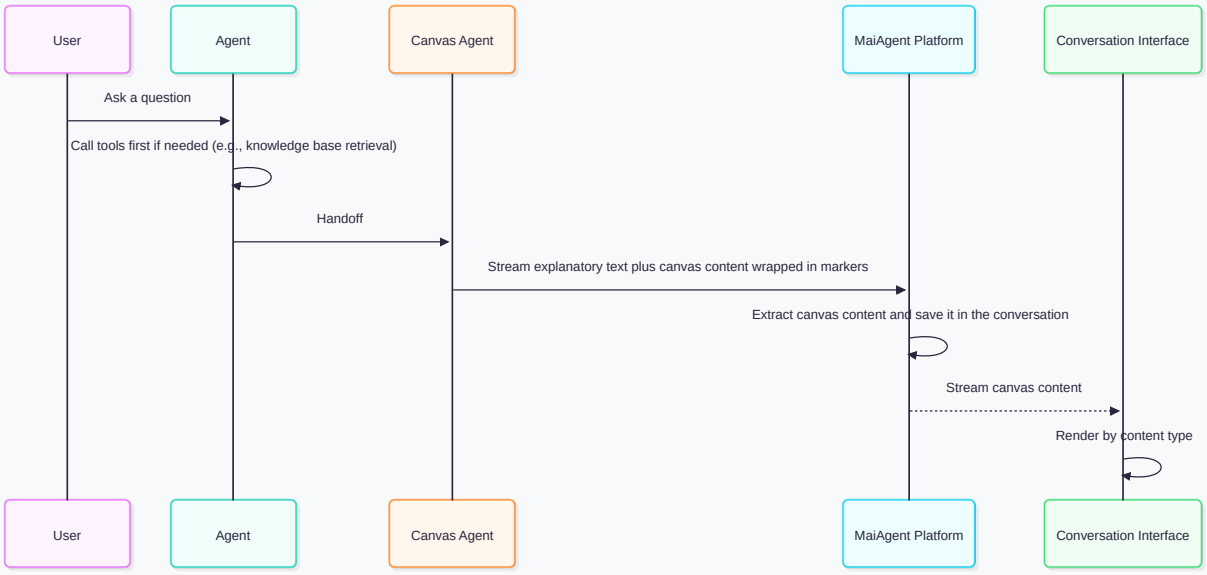
How It Works in MaiAgent

Configure and Enable Canvas

Agent: In the Agent’ s Response Mode Settings, set Response Mode to Agent Mode. Then select Canvas Mode (Canvas) under Agent Mode below. Only models that support tool calling can use canvas mode. Your organization’ s plan may also limit the available Agent modes.

MaiGPT: Uses canvas mode by default. Members can use it directly in conversations.

The Flow of a Response



Mermaid 圖表

Handoff: An Agent in canvas mode has an additional handoff tool, and its system prompt includes handoff instructions. To draw diagrams, visualize data, create web pages or UI components, or generate formatted documents, it hands off to the canvas agent. Maps are an exception: if the Agent has already obtained coordinates using the place search tool, it follows the prompt to output a map canvas itself.

Generation: The canvas agent uses the same model as the Agent and generates content using the platform's canvas prompt. Data retrieved from the knowledge base during this turn is also included in its prompt. Its tool calling rounds are subject to the Agent's "Maximum Iterations" limit as well.

Markers: Each canvas includes `name` (an identifier; reusing the same name updates the same canvas), `type` (content type), `title` (title), and `display` (display location).

Storage: The platform saves canvas content with the conversation, so you can still view it when you reopen the conversation.

Supported Content Types

`type`	Presentation	Download Format
`markdown`	Markdown document	`.md`
`html`	A single-file HTML page (HTML, CSS, and JS in one file), executed in an iframe	`.html`
`svg`	Vector graphic	`.svg`
`mermaid`	Flowcharts, sequence diagrams, Gantt charts, and other diagrams; rendered after content streaming finishes	`.md` (download links on other conversation platforms use `.mmd`)
`react`	React components compiled and previewed in the browser; supports React, lucide-react, recharts, shadcn/ui components, and Tailwind classes	`.jsx`
`map`	Interactive map with JSON containing a center point, place markers, and routes	`.json`

The canvas panel offers a preview, a source code view, and downloads in the original format or as PDF.

Display Location

`display`	Purpose
`inline`	Embedded directly in conversation messages for small charts or illustrations that help explain the current content

`display`	Purpose
`panel` (default)	Opens in a side panel for outputs users will save, revise, or use outside the conversation, such as documents, complete web pages, interactive components, and maps; also used for longer content (roughly more than 40 lines)

Canvases render only in the conversation interfaces of Web Chat and the MaiGPT web app. Other conversation platforms, such as LINE, Microsoft Teams, and Slack, cannot display canvases. The platform appends canvas download links after the response; these links remain valid for 7 days.

When Canvas Is Used

The handoff conditions and prompt guidelines for the canvas agent are:

Handoff occurs: When users ask to draw diagrams (e.g., SVG), visualize data or concepts (e.g., Mermaid), create web pages or UI components, generate formatted documents or reports, or display places on a map.

Canvas is generally avoided: For explanations or tutorials, suggestions about existing content, content that only makes sense within the current conversation, and one-off questions. When content can be presented directly in the conversation, the prompt instructs the model to do so.

One canvas per message: Each response produces only one canvas unless the user requests otherwise.

To request a canvas, state it explicitly in your question, for example, “Use Canvas to create a…”.

Map Canvas

The canvas agent receives the place search (`place_search`) and public transit directions (`get_transit_directions`) tools only when a Google Maps API key is configured in the deployment environment. Without these tools, it can only mark well-known places based on the model’s existing knowledge.

Each call to these tools consumes Credits. If a MaiGPT member’s role does not allow the corresponding official tools, the canvas agent will not receive them either.

Main Use Cases

Use Case	Common Types
Flowcharts, organizational charts, sequence diagrams, Gantt charts	`mermaid`
Documents such as reports, Email, and meeting minutes	`markdown`
Charts and dashboards using data retrieved from a knowledge base or database	`react`
Single-page websites and interface prototypes	`html`, `react`
Place introductions, itinerary planning, and transportation routes	`map`

Use Case	Common Types
Simple icons and illustrations	`svg`

For creating reports from database query results, see Text to SQL Report Creation (Canvas Mode) in the User Manual.

Demo Video

📺 **預覽連結：**https://drive.google.com/file/d/1GW7o3m130M-yESZ4h6VwZvNXDhdJDiFe/view?usp=drive_link

Limitations and Considerations

Role instructions are not passed to the canvas agent: Its system prompt consists of the platform’s canvas prompt and this turn’s knowledge base data, without the Agent’s role instructions. It can see the full conversation, but formatting, terminology, or branding requirements in role instructions may not be reflected in the canvas content. Include them directly in your question when needed.

The canvas prompt is configured at the platform level: All organizations share the same canvas prompt and cannot customize it themselves.

HTML and React limitations: HTML canvases in Web Chat run in a sandboxed iframe (only script execution is allowed, and the iframe has a different origin from the host website). They cannot access the host website’s cookies or storage. The canvas prompt instructs the model not to use images from the web and to load external scripts only from `cdnjs.cloudflare.com`. React canvases can only import the packages listed in the table above.

Correct rendering is not guaranteed: The platform does not check the syntax of generated canvas content. If the model generates invalid code or diagram syntax, the content may fail to display. You can ask the Agent to fix it and regenerate it.

Making changes: Markdown canvases cannot be edited directly on screen. To make changes, describe them in the conversation. The canvas prompt instructs the model to regenerate the complete content using the same `name`. Alternatively, download and edit the source file yourself; local edits are not written back to the canvas.

Other conversation platforms receive download links only: See Display Location above.

See the User Manual for Instructions

Canvas in MaiGPT: View, download, and ask AI to modify canvases

Configure Tools for an Agent: Switch the Agent’s Response Mode

Text to SQL Report Creation (Canvas Mode)

AI Safety Guardrails

MaiAgent provides several safeguards along the path where content enters and leaves large language models. The system enforces them without relying on the model's self-discipline. This page explains who configures each mechanism, where it applies, and when to use it. Refer to the user manual for steps in the admin console.

Mechanism Overview

Mechanism	Who Can Configure It	Where It Applies	Suitable Scenarios
Hook Message Interception	Members with “Hook permissions” configure it themselves in the admin console	Before content enters the model and before AI responses are sent	Masking personal data, blocking prompt injection and inappropriate content, filtering sensitive words, and screening responses
ICAP Content Validation	MaiAgent configures it based on the organization's needs	Conversation messages and attachments, and knowledge base files (see that page for scope)	Enterprises with existing ICAP antivirus DLP services that need to integrate existing security controls
AWS Bedrock Guardrails	Only MaiAgent can configure it for an Agent	User input and AI responses	Scenarios requiring Bedrock Guardrails policies (content filtering, denied topics, personal data masking, and grounding checks)

The three mechanisms are configured and executed independently. There is no shared policy layer or unified table of “block
alert
allow” rules. Hooks rewrite or intercept content, Bedrock Guardrails blocks or masks it according to its policies, and ICAP allows or rejects files based on scan results. When multiple mechanisms are enabled, content passes through each check separately. If any mechanism blocks the content, it will not be sent to the model or the user. The platform also does not integrate third-party runtime prompt injection protection products (such as WAFs or AI firewalls). If these controls are needed, place them in the network layer between users and MaiAgent, or use Hook prompt injection detection instead.

System prompts are not security controls. A system prompt can guide an Agent's tone and response scope, but it is only text sent to the model. The model may be induced to ignore it, and it cannot block passages that have already been retrieved. To ensure that certain content never enters the model or appears in a response, use the mechanisms on this page. To restrict which documents users can retrieve, use Query Metadata. For guidance on writing system prompts, see the user manual's System Prompt Design Guide.

Hook Message Interception

Hooks are a protection layer that customers can configure themselves. Create a Hook from a template, adjust its parameters, and apply it to an Agent. An Agent can have multiple Hooks, executed in order.

Prerequisites: The platform must have the Hook feature enabled, and the organization must have access to Hooks (included automatically in enterprise plans, or enabled individually by MaiAgent). If the Hook menu is not visible in the admin console, contact your MaiAgent representative.

Trigger Points and Actions

Trigger Point	Content Checked
Before content enters the model	User messages, attachment content, passages retrieved from the knowledge base, and tool results in Agent mode
Before AI responses are sent	Responses generated by the Agent

There are two actions: **rewrite** (for example, mask personal data before sending) and **intercept** (block the content and reply with a custom message). When attachment content, a knowledge base passage, or a tool result is intercepted, only that content is excluded from the model; the conversation continues.

Detection Methods

Method	Description	Data Destination
Rule matching	Match patterns or keywords, such as email addresses, mobile phone numbers, national ID numbers, credit card numbers, or specified prohibited words	Processed within the MaiAgent platform
Large language model assessment	Use a selected model to assess content, such as whether it is a prompt injection attack or whether a response lacks grounding	Sent to the selected model

Method	Description	Data Destination
Personal data detection service	Detect personal data without a fixed format, such as names and addresses; Google Cloud DLP is currently available	Sent to the detection service
Content safety classification service	Classify content by categories such as hate, harassment, sexual content, violence, self-harm, illegal activities, and jailbreak attacks; OpenAI Moderation is currently available	Sent to the classification service

Personal data detection and content safety classification deduct credits based on the number of characters scanned (per thousand characters). For available templates, each template's parameters, and interception records, see Hook Message Interception, Add and Configure a Hook, and Hook Execution Logs in the user manual.

AWS Bedrock Guardrails

AWS Bedrock Guardrails is a protection service provided by AWS. MaiAgent can assign a Guardrail (ID and version) to an individual Agent. That Agent's conversations then pass through the Guardrail checks:

User input: Checked before entering the model. If a blocking rule is triggered, the Guardrail's configured blocking message is returned directly.

AI responses: Checked before sending, together with the question and retrieved passages. If a blocking rule is triggered, the response is replaced with the blocking message. If a masking rule is triggered, the masked content is returned instead.

Usage limitations:

Only MaiAgent can configure it: There is no configuration option in the admin console. The Guardrail must be created in the AWS environment called by the MaiAgent platform. A Guardrail created in the customer's own AWS account cannot be applied directly. Contact your MaiAgent representative if needed.

Responses are not streamed token by token: For an Agent with a Guardrail applied, the entire response is sent after checks are complete.

Connectivity to AWS Bedrock is required; offline on-premises environments cannot use it.

For scenarios that do not require AWS, prioritize Hooks, which you can configure yourself.

OAuth 2.0 Integration and Automatic Token Refresh

This document explains how the MaiAgent platform integrates the OAuth 2.0 authentication protocol and provides automatic token refresh to ensure secure and continuous access to third-party service integrations.

1. Why Is OAuth Integration Needed?

As an enterprise Agent platform, MaiAgent needs deep integration with various third-party services, such as Google Workspace, Salesforce, and GitHub. OAuth 2.0 provides a standardized, secure authorization mechanism that addresses the following key challenges:

Consideration	Traditional Password Storage	OAuth 2.0 Authorization
Security	Requires storing user credentials, creating a risk of exposure	Uses short-lived access tokens without storing user passwords
Permission Control	Usually grants full account access, without granular control	Allows precise control over authorization scopes, such as read-only Gmail access or calendar management only
Revocation	Requires changing the password to revoke access	Allows access for a specific application to be revoked at any time without affecting other services
User Experience	Requires entering a password on a third-party platform	Completes authorization through a trusted native interface for a smoother experience

Common use cases:

Tool Connector: Agents access services such as Google Drive, Notion, and Slack through OAuth authorization to perform operations such as reading files and sending messages

Data Synchronization: Automatically synchronizes customer data from CRM systems such as Salesforce without manual imports

Automated Workflows: Integrates project management tools such as Jira and Asana so Agents can help create tasks and update statuses

2. MaiAgent OAuth Integration Architecture



Mermaid 圖表

2.1 Authorization Flow Optimizations

MaiAgent implements the following key optimizations:

State Validation: Generates a unique state parameter for each authorization request to prevent CSRF attacks

Member-Level Authorization: Supports individual OAuth authorization for organization members, enabling more granular permission management

Flexible Callback Handling: Makes the connector_id parameter optional to support organization-level or member-level authorization

Access Token Validation: Requires the presence of access_token in token responses to ensure the authorization flow is complete

2.2 Secure Token Storage

MaiAgent uses multiple layers of security to protect OAuth tokens:

Encrypted Storage: Encrypts all tokens (Access Tokens and Refresh Tokens) before storing them

Client Credential Management: Stores the OAuth application's client_id and client_secret for token refresh

Token Metadata: Records token expiration times, authorization scopes, and associated user and connector information

Legacy Token Handling: Automatically excludes legacy tokens without client credentials, ensuring the system uses only tokens that can be refreshed automatically

2.3 Automatic Token Refresh

Access tokens typically have a short lifetime. MaiAgent obtains new tokens before expiration in two ways, so Agents and connectors do not need to ask users to authorize again each time: **scheduled refresh** and **refresh on use**.

sequenceDiagram participant Scheduler as Scheduled Task participant DB as Token Database participant OAuth as OAuth Provider loop Every 20 minutes Scheduler->>DB: Query tokens that have expired or expire within 30 minutes,
and have been active in the last 7 days (up to 50 per run) DB-->>Scheduler: List of tokens to refresh loop Process each token Scheduler->>OAuth: Exchange Refresh Token for a new Access Token alt Refresh succeeds OAuth-->>Scheduler: New token information Scheduler->>DB: Update token and expiration time else Authorization server returns 400/401 Scheduler->>DB: Disable this authorization (reauthorization required) else Other error Scheduler->>DB: Keep unchanged and retry at the next scheduled run end end end

Mermaid 圖表

Scheduled Refresh Rules

Item	Rule
Frequency	Once every 20 minutes
Tokens to Refresh	Tokens that have expired or will expire within 30 minutes
Coverage	Member-authorized connectors, member-authorized MCP tools, and contact-authorized connectors
Activity Requirement	Member tokens: updated within the last 7 days, with both the member and their organization still active; contact tokens: used within the last 7 days (authorization time is used for tokens that have never been used)
Prerequisites	The token has a Refresh Token and a stored OAuth Client ID; disabled connectors or tools are skipped
Batch Size	Up to 50 tokens, processed in expiration order; only one refresh task runs at a time
Individual Refresh Failure	Logs the error without retrying during the same run; retries at the next scheduled run if the token still meets the requirements
Entire Task Failure	Retries after a fixed 60-second interval, up to 3 times
Authorization Server Rejection (HTTP 400/401)	Disables this authorization; the user must authorize again
No New Refresh Token in the Response	Keeps the original Refresh Token

Tokens inactive for more than 7 days are not refreshed on a schedule. A member's tokens also stop being refreshed after the member is deactivated or leaves the organization. Subsequent use follows the “Refresh on Use” process below; if the Refresh Token is no longer valid, reauthorization is required.

Refresh on Use

When an Agent needs a token that has expired or will expire within 60 seconds, the platform obtains a new token before making the call. If refresh fails, the call cannot obtain a token. After a connector's authorization fails to refresh, further attempts are paused: the delay starts at 60 seconds and doubles with each failure, up to 1 hour. MCP tool tokens (member-authorized or shared across an organization) are also refreshed before use under the same 60-second rule.

2.4 Member-Level Authorization Support

MaiAgent now supports more granular member-level OAuth authorization:

Independent Authorization Flow: Each organization member can complete OAuth authorization independently, without an organization administrator's involvement

Authorization Metadata: Includes member-related context in OAuth state for accurate tracking of the authorization flow

Simplified Redirect URIs: Optimizes the OAuth callback URI structure for greater integration flexibility and maintainability

Frontend State Management: Uses sessionStorage to manage member OAuth redirects and maintain continuity throughout the authorization flow

3. OAuth Integration Best Practices

3.1 Authorization Scope Design Principles

Follow the principle of least privilege when configuring OAuth applications:

Request Only Required Permissions: If you only need to read Google Calendar, do not request Gmail write permissions

Clearly Inform Users: Explain why these permissions are needed before authorization

Support Incremental Authorization: Guide users through authorization again when additional permissions are needed

3.2 Token Lifecycle Management

MaiAgent follows these token management best practices:

Access Token: Short-lived (usually 1 hour), used for actual API calls

Refresh Token: Long-lived (from several months to potentially indefinite), used to obtain new Access Tokens

Automatic Refresh Timing: Checks every 20 minutes and refreshes tokens expiring within 30 minutes; also refreshes before use if expiration is within 60 seconds. See 2.3 for the rules

Invalid Refresh Token: When the authorization server rejects a refresh, the authorization is disabled and the user must authorize again

3.3 Security Considerations

HTTPS Required: All OAuth-related communications must use HTTPS

State Parameter Validation: Prevents CSRF attacks and ensures the integrity of authorization requests

Encrypted Token Storage: Uses industry-standard encryption algorithms to protect sensitive data

Regular Audits: Records all token usage to support security reviews

4. Technical Advantages of MaiAgent OAuth Integration

4.1 Development and Integration Advantages

Standardized Interface: Follows the OAuth 2.0 standard and is compatible with most third-party services

Simplified Configuration: Administrators only need to provide basic OAuth application information to complete integration

Automated Maintenance: Token refresh is fully automated and requires no manual intervention

Flexible Deployment: Supports multiple authorization modes at the organization and member levels

4.2 Security and Compliance Advantages

No Password Storage: Completely avoids the security risks of storing user passwords

Granular Permission Control: Allows different authorization scopes for different services

Immediate Revocation: Administrators or users can revoke authorization at any time, with changes taking effect immediately

Audit Trail: Records all authorization and token usage activity in full

4.3 User Experience Advantages

Seamless Refresh: Automatic token refresh is transparent to users and does not interrupt their experience

Smooth Authorization: Completes authorization through a trusted native interface without concerns about password exposure

Continuous Availability: Services can continue operating as long as the Refresh Token remains valid

5. Related Technical Documentation

Remote MCP Service Overview - Learn how to integrate Remote MCP services through OAuth

Composio Integration - OAuth integration examples for the Composio platform

Zapier Integration - OAuth integration guide for the Zapier platform

References

OAuth 2.0 RFC 6749

OAuth 2.0 Security Best Current Practice

PKCE for OAuth Public Clients (RFC 7636)

ICAP Content Validation Integration

This document explains how the MaiAgent platform integrates ICAP (Internet Content Adaptation Protocol) to scan and validate user-uploaded files and conversation content for security, protecting against malware and inappropriate content.

1. What Is ICAP?

ICAP (Internet Content Adaptation Protocol) is a lightweight protocol for forwarding content to external services for processing, such as virus scanning, content filtering, and data loss prevention.

1.1 Core Value of ICAP

Consideration	Without Content Validation	With ICAP Content Validation
Security	Users may upload malware or inappropriate content	Automatically scans and blocks malicious files and content
Compliance	Difficult to meet enterprise security policy requirements	Integrates enterprise antivirus and DLP solutions
Flexibility	Requires custom content detection logic	Uses a standard protocol to integrate specialized third-party services
Maintainability	Requires ongoing updates to virus signatures and detection rules	Specialized vendors maintain the detection engine

1.2 Common Use Cases

File Upload Security: Documents uploaded to the knowledge base must undergo virus scanning first

Conversation Content Filtering: Detects inappropriate language and sensitive information in conversations

Data Loss Prevention (DLP): Prevents users from uploading files containing confidential information

Compliance Checks: Ensures content meets regulatory requirements such as GDPR and HIPAA

2. MaiAgent ICAP Integration Architecture

flowchart TB subgraph Client["Client"] User["User"] WebChat["Web Chat Interface"] end subgraph MaiAgent["MaiAgent Platform"] API["Upload API"] Queue["Processing Queue"] ICAP_Client["ICAP Client"] DB[("Database")] Storage[("File Storage")] end subgraph ICAP_Service["ICAP Service"] Scanner["Antivirus Engine"] DLP["DLP Engine"] Filter["Content Filter"] end User -- "Upload File" --> WebChat WebChat -- "HTTP POST" --> API API --> Queue Queue --> ICAP_Client ICAP_Client <-. "ICAP REQMOD" .-> Scanner ICAP_Client <-. "ICAP RESPMOD" .-> DLP ICAP_Client <-. "ICAP OPTIONS" .-> Filter ICAP_Client -- "Validation Passed" --> Storage ICAP_Client -- "Record Results" --> DB ICAP_Client -. "Validation Failed
Send Event" .-> WebChat WebChat -. "Display Error" .-> User

Mermaid 圖表

2.1 Core Components

ICAP Client: MaiAgent's built-in ICAP protocol implementation, responsible for communicating with the ICAP server

ICAP Server: A third-party content scanning service, such as an antivirus gateway from Symantec, McAfee, or Trend Micro

Processing Queue: Handles file uploads and scanning tasks asynchronously to avoid blocking user operations

Scan Records: Stores all scan results, including file hashes, scan times, and outcomes

2.2 ICAP Request Flow

sequenceDiagram participant User as User participant WebChat as Web Chat participant MaiAgent as MaiAgent API participant ICAP as ICAP Server participant Storage as File Storage User->>WebChat: Upload File WebChat->>MaiAgent: POST /upload MaiAgent->>MaiAgent: Generate File ID Note over MaiAgent,ICAP: ICAP REQMOD Request MaiAgent->>ICAP: REQMOD /scan HTTP/1.1 Encapsulated: req-hdr=0, req-body=... ICAP->>ICAP: Scan File Content alt File Is Safe ICAP-->>MaiAgent: 200 OK (File Passed) MaiAgent->>Storage: Store File Storage-->>MaiAgent: Storage Successful MaiAgent-->>WebChat: 200 OK {fileId: ...} WebChat-->>User: Upload Successful else File Has Issues ICAP-->>MaiAgent: 403 Forbidden (Virus/Prohibited Content) MaiAgent->>WebChat: Socket.IO: ICAP_BLOCKED Event WebChat-->>User: Display Error Message MaiAgent->>MaiAgent: Record Scan Failure else ICAP Server Error ICAP-->>MaiAgent: 500 Internal Server Error MaiAgent->>WebChat: Socket.IO: ICAP_ERROR Event WebChat-->>User: System Error, Please Try Again Later MaiAgent->>MaiAgent: Write Error Log end

Mermaid 圖表

3. ICAP Protocol Implementation Details

3.1 ICAP Request Format

MaiAgent uses REQMOD (Request Modification) mode to send files to the ICAP server:

```
REQMOD icap://icap-server.example.com/scan ICAP/1.0



POST /upload HTTP/1.1



400
[Binary file content in chunked encoding]
0
```

Key Fields:

REQMOD: Request modification mode, used to inspect content before storage
Encapsulated: Describes the encapsulation format of the HTTP request
Allow: 204: Tells the ICAP server it may return 204 directly if no content modification is needed
Chunked Encoding: Sends file content using chunked transfer encoding

3.2 Chunked Transfer Encoding

MaiAgent correctly implements HTTP chunked transfer encoding:

```
Chunked Transfer Format:
\r\n
\r\n
 ...
0\r\n\r\n  (End marker)
```

Why Is Correct Chunked Encoding Necessary?

ICAP requires HTTP chunked transfer encoding to transmit request and response bodies
Incorrect encoding prevents the ICAP server from parsing file content correctly

MaiAgent corrected the hexadecimal representation of chunk sizes to ensure compatibility with the ICAP standard

3.3 ICAP Response Handling

The ICAP server may return the following responses:

Status Code	Meaning	MaiAgent Handling
200 OK	Content has been inspected or modified	Accepts the modified content (if any) and stores the file
204 No Content	No content modification is needed	Stores the original file directly
403 Forbidden	Content is prohibited (for example, it contains a virus)	Rejects storage and sends an ICAP_BLOCKED event to the client
500 Internal Error	ICAP server error	Logs the error and sends an ICAP_ERROR event to the client

4. WebChat Integration

When the ICAP scan completes, the system immediately notifies the user of the result:

File Blocked: Displays a user-friendly error message explaining why the file was rejected (for example, a virus was detected)

System Error: Prompts the user to try again later and writes an error log

User Experience Improvements

Immediate Feedback: Displays a "Scanning" status immediately after a file is uploaded

Progress Indicator: Displays a progress bar during large file uploads

User-Friendly Error Messages: Explains why a file was rejected in easy-to-understand language

Retry Mechanism: Offers an option to upload again if the error is temporary

5. Scan Record Management

5.1 Record Storage

MaiAgent stores detailed records of all ICAP scans:

Each scan record contains the following information:

Field	Description
File ID	Unique identifier of the scanned file
File Name	Original file name
File Hash	SHA-256 hash used to verify file integrity
Scan Result	PASS (passed), BLOCKED (blocked), ERROR (error)
Scan Reason	Specific reason recorded when a file is blocked
ICAP Server	Address of the ICAP server that performed the scan
Scan Time	Timestamp of the scan
User/Organization	User who initiated the scan and their organization

5.2 Expired Record Cleanup

MaiAgent implements an automatic cleanup mechanism that periodically deletes expired scan records:

The system automatically removes scan records that exceed the retention period through a scheduled background task.

Cleanup Policy:

Retention Period: Uses the organization's ICAP "Record Retention Days" setting, which defaults to 30 days; 0 means records are retained permanently

Scope: Retention is configured per organization, and the same period applies to all scan records within that organization

6. Configuration and Deployment

6.1 ICAP Server Configuration

MaiAgent configures ICAP per organization through system administration; the management console has no configuration screen. To enable or adjust ICAP, contact your MaiAgent representative and provide the ICAP server information. Each organization can configure the following settings:

Setting	Description
ICAP Server URL	Connection URL of the ICAP service; the connection can be tested during configuration
Enabled	Scanning runs only when enabled. MaiAgent must also enable ICAP checks for the model used by the Agent; both settings must be enabled for checks to take effect

Setting	Description
Block Message	Message shown to users when content is intercepted
Record Retention Days	Retention period for scan records; defaults to 30 days, with 0 meaning permanent retention

6.2 Supported ICAP Services

MaiAgent is compatible with the following major ICAP services:

Symantec Protection Engine: Enterprise antivirus and content filtering
McAfee Web Gateway: Gateway-level content scanning
Trend Micro IWSVA: Integrated web security and content scanning
Kaspersky Scan Engine: Kaspersky's ICAP scanning engine
ClamAV (via c-icap): Open-source antivirus solution

6.3 Performance Considerations

To maintain system performance, we recommend:

Asynchronous Scanning: Scan large files through background tasks without blocking upload responses
Caching: Cache scan results for identical files (with the same hash)
ICAP Connection Pool: Maintain persistent connections to the ICAP server to reduce connection overhead
Load Balancing: Distribute the scanning load across multiple ICAP server instances

7. Technical Advantages of MaiAgent ICAP Integration

7.1 Security Advantages

Multilayer Protection: Scans files before storage to guard against malicious content
Immediate Blocking: Blocks threats as soon as they are detected, preventing storage in the system
Specialized Engines: Integrates industry-leading antivirus and DLP engines
Complete Records: Keeps detailed records of all scanning activity for security audits

7.2 Compliance Advantages

Standard Protocol: Uses the industry-standard ICAP protocol for easy integration with existing enterprise security infrastructure
Audit Trail: Complete scan records meet compliance requirements
Flexible Configuration: Adjusts scanning policies to meet the compliance needs of different industries

Data Protection: Supports DLP engines to prevent sensitive data leaks

7.3 Operational Advantages

Decoupled Architecture: Separates ICAP scanning logic from the application for easier maintenance

Horizontal Scaling: Easily adds ICAP servers to handle more scanning requests

Centralized Management: Enterprises can use existing ICAP infrastructure to manage content scanning across all applications

Detailed Logs: Complete scan logs support troubleshooting and performance optimization

8. Related Documentation

AI Security Guardrails - Learn about MaiAgent's multilayer security strategy

File Upload and Knowledge Base Management

Reference Links

ICAP RFC 3507

Chunked Transfer Encoding (RFC 7230)

Open ICAP Forum

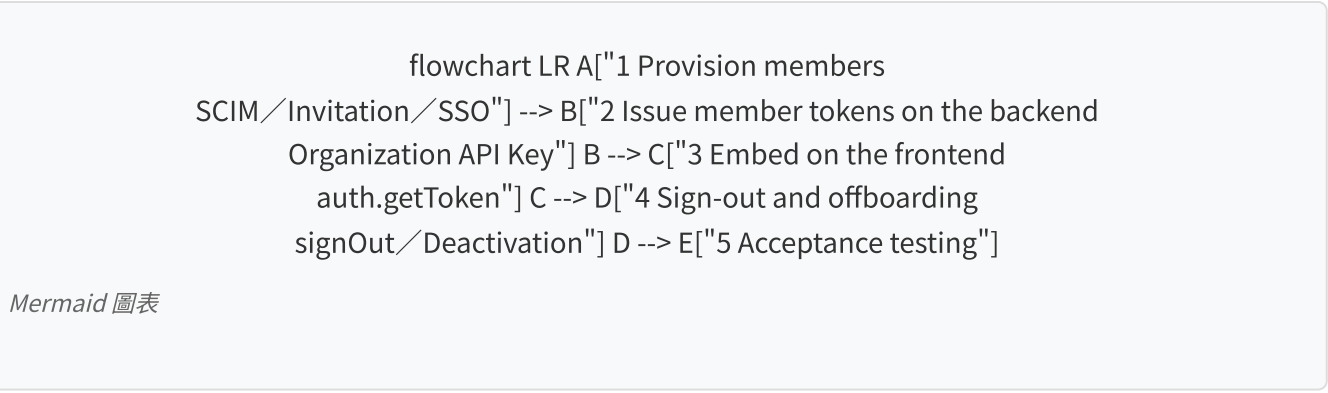
MaiAgent Work｜Employee Productivity Integration

Add MaiGPT to Your System

This page walks engineers through adding MaiGPT to their system from start to finish. Whether you run your company's portal, an internal workspace, or a SaaS platform managed for customers, the process is the same as long as users are **members of a MaiAgent organization** (employees or staff responsible for handling work). Each step explains what to do here; follow the links when you need more detail.

> If your users are your customers, registered users, or anonymous visitors rather than organization members, they cannot use the MaiGPT product itself. Instead, use a custom AI Agent with MaiGPT window mode. For contact identity integration, see the Software Vendor Integration Guide.

Overview



Mermaid 圖表

Once complete, employees log in to your system and see MaiGPT directly on the page. They can generate images, use Deep Research, and access connectors. Permissions, quotas, and conversation history follow their MaiAgent member identity. They do not need to log in to MaiAgent separately or even know that MaiAgent exists.

Prerequisites

Item	Where to get it
A MaiAgent organization with a plan that includes MaiGPT and identity synchronization	Contact sales if you are unsure
An organization API Key whose holder is the organization owner or has administrative permissions for “Organization Overview”	See Quickstart for instructions

Item	Where to get it
The MaiGPT Web Chat ID	The entry in the `GET /api/v1/web-chats/` response with the name `MaiGPT` and `accessType` set to `web`
Roles created and permissions configured in advance	In the admin console, go to "Organization Settings → Organization Overview → Edit Role Permissions," create a role such as "Employee," and enable MaiGPT access (configuration instructions). **All member capabilities come from roles**. Without this step, the rest of the setup will not work
Decide where members come from	Choose one of the three options in the next section

Step 1: Provision Members

Your users must first be members of the MaiAgent organization. Choose whichever of these three methods is easiest:

> Does anyone in your organization already use MaiAgent? Connecting SCIM will neither invalidate their existing accounts nor create duplicates. Existing members taken over by SCIM will subsequently stay synchronized with your system. See What Happens to Existing Accounts and Members for details.

Your system has user management (recommended)

Call MaiAgent's SCIM endpoints from your system's user creation, update, and deactivation events to keep members synchronized in real time.

In the admin console, select SCIM under "Organization Settings → Identity Synchronization," then copy the Endpoint URL and Token from the "Connection Information" tab (the Token is displayed only once; store it in server environment variables)

Add the `MaiAgentMembers` client from the Real-Time Synchronization page to your project and call it from your events:

```
members = MaiAgentMembers()   # Read environment variables MAIAGENT_SCIM_URL
MAIAGENT_SCIM_TOKEN

# Create a user
user.maiagent_member_id = members.on_user_created(
    external_id=user.id, email=user.email, name=user.display_name, groups=
["Employee"],
)
# Change name or department
members.on_user_updated(user.maiagent_member_id, user.id, user.email,
user.display_name, ["Employee"])
# Suspend access or offboard
members.on_user_deactivated(user.maiagent_member_id)
```

On initial launch, run `on_user_created` once for all existing users (idempotent; see Backfill), then maintain synchronization through events.

You only have a roster (HR export, Excel, database)

Use the Roster Synchronization Tool: prepare a CSV or JSON file with four fields (personnel ID, Email, name, role), configure the SCIM connection environment variables, and review the plan with `--dry-run` before running it for real. Then schedule it to run daily. Users on the roster are created or updated; users removed from it are deactivated.

You have Okta／Entra ID／JumpCloud

Enter the SCIM Endpoint URL and Token in your IdP's provisioning settings. The IdP will automatically push users and groups. See Push from an IdP for details.

Remember these three points—the most common stumbling blocks in real integrations:

Match role names to the roles created in the admin console: the names in `groups` when SCIM creates a member must match roles with permissions already configured in the admin console. An unmatched name automatically creates a new role with **no permissions**.

Email is the account key: `userName` is the Email used later to issue tokens. After creation, Email cannot be changed through SCIM.

Accounts created through SCIM have no password: employees using this approach do not log in to MaiAgent, which is exactly the intended behavior. To also allow access to the MaiGPT web interface or App, configure SSO separately.

Step 2: Issue Member Tokens on the Backend

Create a backend endpoint that only **logged-in users** can call (for example, `GET /maigpt-token`). Internally, use the organization API Key to request a short-lived token for that member from MaiAgent:

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
  -H "Authorization: Api-Key <Your organization API Key>"
  -H "Content-Type: application/json"
  -d '{"email": "alice@acme.com"}'
```

A successful request returns `201`:

```
{
  "accessToken": "<Short-lived member JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": { "id": "c1a4 ... ", "userId": "5e77 ... ", "email": "alice@acme.com",
"name": "Alice" }
}
```

Return the entire response to the frontend unchanged. Rules and status codes:

Fact	Explanation
Valid for 15 minutes; no refresh token is issued	The frontend SDK handles renewal automatically (Step 3). Your backend only needs to issue a token whenever called—**always issue a new one; do not cache it**
The token works only with MaiGPT conversation endpoints	Calling any API under `/api/v1/` with it returns `401`, so a leaked token cannot access organization data
`401` / `403`	Api-Key was not used, or the Key holder is neither the owner nor an organization administrator
`404`	This Email does not belong to an active member of this organization—Step 1 was not completed, or the member has been deactivated
Every token issuance creates an audit record	Available in the organization's API request logs; the token in the response is not recorded

For parameters and more details, see Member Token Issuance.

Step 3: Embed on the Frontend

Add the embed code to the page where you want MaiGPT and point `auth.getToken` to the endpoint from Step 2. The SDK handles everything else (fetching a token at initialization, automatically fetching another before expiration, and continuing the same conversation):

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',            // Optional: brand name in the upper-left
corner
    locale: 'zh-TW',                    // Optional: interface language
    auth: {
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' })
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

Set sourceId to member.userId from the response, not member.id or the user ID in your system. MaiGPT uses it to associate conversations with the member. An incorrect value results in anonymous conversations and prevents access to that member's history.

Set the container height yourself. For an SPA, load the SDK after the container mounts. The `baseUrl` and SDK URL are SaaS addresses; replace them with your domain for a private deployment. For layout and other parameters, see Embedding MaiGPT Mode.

Step 4: Sign-Out and Offboarding

Event	What you need to do	Result
A user logs out of your system	Call `MaiAgent.auth.signOut()` on the frontend	The iframe clears the identity and stops renewal; issued tokens remain valid for up to 15 more minutes
A user leaves the organization or has access suspended	Deactivate them through Step 1 (`on_user_deactivated`, roster removal, or IdP deactivation)	Subsequent token requests return `404`, and the embedded page loses access; conversation and quota records are retained
A user returns to work	Reactivate them through Step 1	Roles and history are restored

Step 5: Acceptance Testing

Check these items in order. Each has a clear pass criterion:

The member exists and has a role: the person appears under “Member Management” in the admin console, assigned to the role whose permissions you configured.

Token issuance succeeds: the curl request in Step 2 returns `201`; using a nonexistent Email returns `404`.

Conversations work: open MaiGPT on the embedded page, send a message, and receive a reply. The conversation is visible when you log in to the MaiGPT web interface as that member (if SSO is available), or in the admin console's conversation records.

Capabilities are correct: ask it to generate an image. If the role enables connectors and Deep Research, their entry points appear.

Renewal is seamless: leave the page open for more than 15 minutes, then send a message. The same conversation continues without refreshing the page.

Sign-out takes effect: messages are rejected after `signOut()`; after deactivating the member, Step 2 returns `404`.

After Launch

SCIM Token and API Key rotation: the admin console reminds you when a SCIM Token is more than 180 days old. Regenerating it immediately invalidates the old token; remember to update the server environment variables.

Plan capacity: member count affects built-in Agent seats and each member's Credits quota. See Member Credits Quota Management.

Auditing: every token issuance appears in the organization's API request logs. Each SCIM synchronization appears under “Identity Synchronization → Synchronization Logs” in the admin console.

Troubleshooting

Symptom	Cause	Resolution
Token issuance returns `404`	Email does not belong to an active member of this organization	Return to Step 1 and confirm that the member exists, is not deactivated, and has the same Email (case-insensitive)
Token issuance returns `403`	The API Key holder is neither the owner nor someone with organization administrative permissions	Use a Key issued by an administrator

Symptom	Cause	Resolution
The chat window shows “Failed to create conversation (400): You do not have permission to use MaiGPT”	The member's role lacks MaiGPT access, or `auth` was not passed	Enable MaiGPT in the admin console's role permissions; confirm that `getToken` returns `accessToken` and `organizationId`
Conversations are anonymous and history is unavailable	Incorrect `sourceId`	Use `member.userId`
Sending messages fails after 15 minutes	`getToken` is not called or fails (check the browser console for `[MaiAgent]` errors)	Confirm that the endpoint is available to logged-in users and returns the correct response format
The member is created but cannot use anything	No role is assigned, or the role has no permissions	Configure role permissions in the admin console; assign the same role name through SCIM
Changing Email has no effect	SCIM does not allow Email changes	Change it in the admin console, or deactivate the old account and create one with the new Email

For more SCIM error codes, see the Roster Synchronization Tool Error Reference.

Further Reading

Member Provisioning: SCIM／Invitation／SSO — Complete SCIM API reference and comparison of the three approaches

Real-Time Synchronization: Call from User Events／Roster Synchronization Tool

Integrating MaiGPT: Choose the Right Agent and Enable Its Capabilities — MaiGPT product vs. custom Agent, and complete parameters for embedding with member identity

Web Chat Embedding and SDK／Embedding MaiGPT Mode

Integrate MaiGPT: Choose the Right Agent and Add Capabilities

"MaiGPT" refers to two different things on the MaiAgent platform. Distinguishing them before integrating can save most of the back-and-forth:

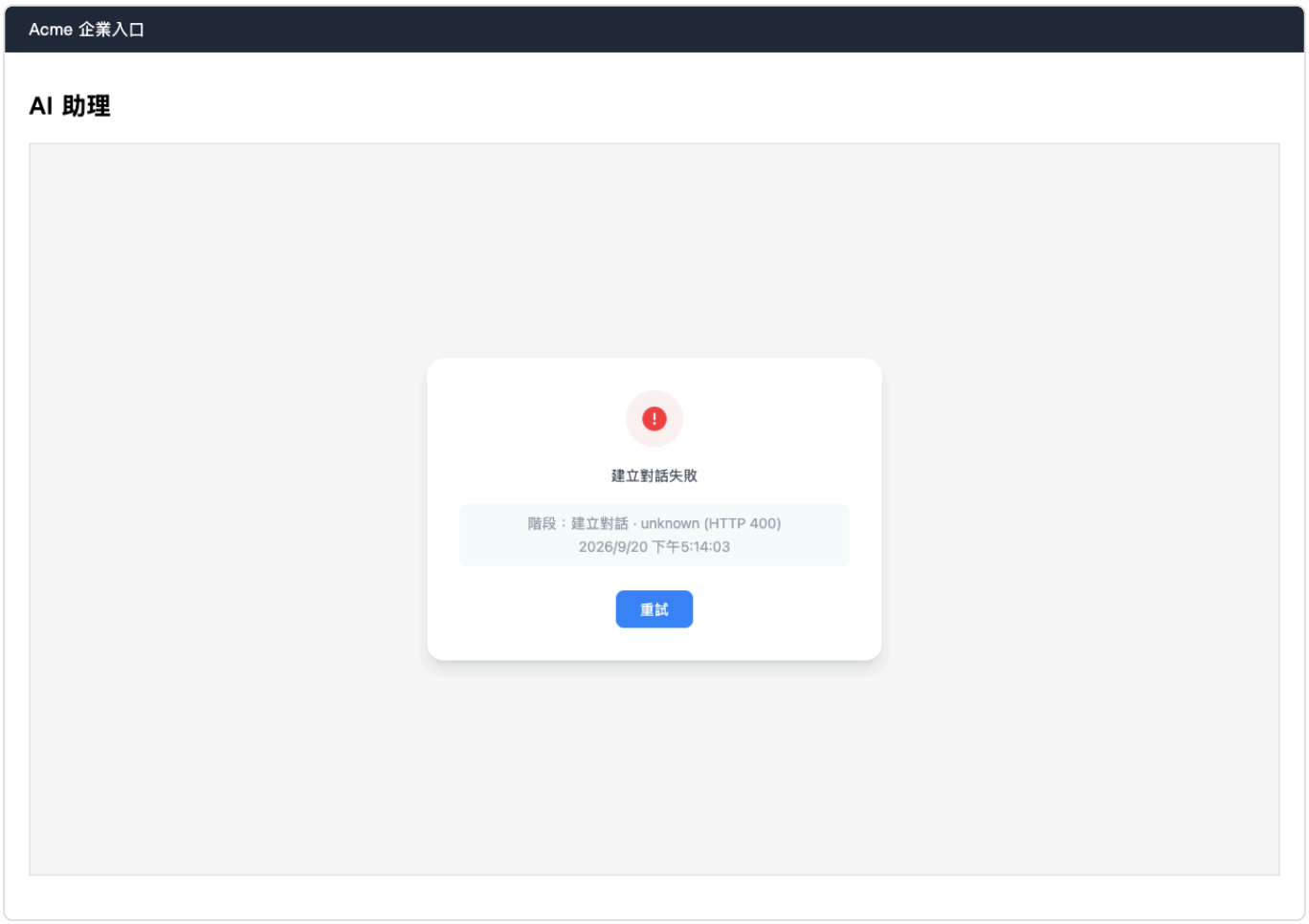
	MaiGPT product (built-in organization Agent)	**MaiGPT window mode (a Web Chat SDK layout)**
What it is	An enterprise AI workspace automatically provided to every organization and centrally maintained by MaiAgent, with built-in AI image generation, Deep Research, connectors, and more. See What is MaiGPT?	A **full ChatGPT-like interface** (sidebar conversation history, search, settings) enabled by `enabledWindowModes: ['maigpt']`; purely a frontend presentation
Who can use it	Organization **members** after signing in (web, iOS App, LINE, Microsoft Teams), with permissions controlled by role usage permissions	Any Web Chat can use this layout, including anonymous visitors and users with contact identity
Can you embed it in your system?	**Only for organization members**: creating a conversation requires member identity and the `maigpt_access` permission. Embedding it directly using its Web Chat ID returns `400 You do not have permission to use MaiGPT.` To embed it for members, follow embedding with member identity and have your backend issue member tokens	**Yes**: sections 2 and 3 of this page explain how
Can you customize it?	**No**: instructions, models, tools, and skills are configured centrally by the platform. Organizations can only **restrict** access through role permissions	**Yes**: the interface runs on an AI Agent you create. You control its instructions, models, tools, and knowledge bases

Conclusion: there are two use cases. For **your customers who are not organization members, create your own AI Agent, add the tools you need, and embed it in MaiGPT window mode** (sections 2 and 3). For **organization members** who need the MaiGPT product's capabilities (image generation, Deep Research, connectors, role permissions), use embedding with member identity (section 5), with your backend issuing short-lived tokens for members.

1. Common Symptoms

What you see	**Cause**	**Solution**
The interface looks like MaiGPT, but responses behave like a regular	You embedded a regular AI Agent, and either (1) its reply mode is not **Agent**	Follow section 2 to switch to Agent mode and add tools

What you see	Cause	Solution
chat. When asked to generate an image, the AI replies “I cannot generate images”	**mode**, or (2) it has no image generation or other tools attached	
The chat window will not open and displays “Failed to create conversation (HTTP 400)”; the API returns `You do not have permission to use MaiGPT.`	The embedded Web Chat ID belongs to the organization's built-in MaiGPT product, and the visitor has no member identity	For customers → use your own Agent's Web Chat ID (section 3); for members → provide a member token as described in embedding with member identity
MaiGPT does not appear in the backend AI Agent list API	`GET /api/v1/chatbots/` lists only Agents created by the organization; the MaiGPT product is excluded	This is expected behavior; integrations use Agents you create



Embedding the organization's built-in MaiGPT Web Chat: the visitor is not a signed-in member, so conversation creation is rejected (HTTP 400)

2. Create an AI Agent That Can Generate Images and Use Tools

The following example uses AI image generation. Add other capabilities (web search, web fetching, MCP tools, your own API tools) in the same way.

Create an AI Agent

In the admin console, go to "AI Features → AI Assistants", create a new AI Agent, and set its role instructions and model. See Create an Agent in the user manual.

Switch the Reply Mode to "Agent Mode"

Open the Agent's "Reply Mode Settings" and change its reply mode to **Agent mode**.

> Only Agent mode calls tools. The default "RAG Q&A" mode responds quickly but does not use any tools—this is the most common reason why an Agent still cannot generate images after you add an image generation tool.

Add Tools

Open the Agent's "Tools" page, click "Select Tools", and select the built-in tools you need, such as:

Image generation: `Global_Image_Tool_OpenAI_GPT_Image_2`, `Global_Image_Tool_Gemini_3`, and others (selecting one is enough; for model differences, see Built-in AI Image Generation Tools)

Web search: `Global_Tavily_Search_Tool`

Web fetching: `Global_Web_Fetch_Tool`

Your system's API tools or MCP tools

For the full procedure, see Configure Tools for an Agent in the user manual.

Create a Web Chat Conversation Platform

Under "Customer Service Conversations → Conversation Platforms", add a **Web Chat** for this Agent and obtain its `webChatId`. See Connect a Conversation Platform: Website.

Automate with the API (for ISVs Setting Up Customer Environments)

All four steps above have corresponding APIs, allowing software vendors to automate provisioning for each customer. For authentication and obtaining an API Key, see Quickstart.

```
# 1. List built-in tools and obtain the image generation tool's id
GET /api/v1/tools/

# 2. Create an AI Agent: agent reply mode + attached tools
POST /api/v1/chatbots/
{
  "name": "Acme GPT",
  "instructions": "You are Acme's AI assistant. Please respond in Traditional
Chinese.",
  "largeLanguageModel": "<LLM id>",
  "embeddingModel": "<embedding model id>",
  "rerankerModel": "<reranker model id>",
  "replyMode": "agent",
  "tools": ["<id of Global_Image_Tool_OpenAI_GPT_Image_2>"]
}

# 3. Create a Web Chat conversation platform for the Agent (channel.id in the
response is webChatId)
POST /api/v1/inboxes/
{
  "name": "Acme GPT",
  "chatbot": "<chatbot id>",
  "channelType": "web",
  "channel": {
    "name": "Acme GPT",
    "theme": { "primaryColor": "#1890ff", "...": "..." },
    "darkTheme": { "primaryColor": "#404040", "...": "..." }
  }
}
```

To change an existing Agent's mode or add tools, use `PATCH /api/v1/chatbots/{id}`. Pass the complete array of tool ids in `tools` (the entire array is replaced)

`theme` and `darkTheme` are required. To get a ready-made set of fields to copy, call `GET /api/v1/web-chats/{existing webChatId}`

Always create a Web Chat together with its inbox via `POST /api/v1/inboxes/`. Calling `POST /api/v1/web-chats/` directly creates an orphan Web Chat without an inbox, which returns 404 when embedded

3. Embed in MaiGPT Window Mode

Use the `webChatId` from section 2 and add the embed code from Embed in MaiGPT Mode to your page:

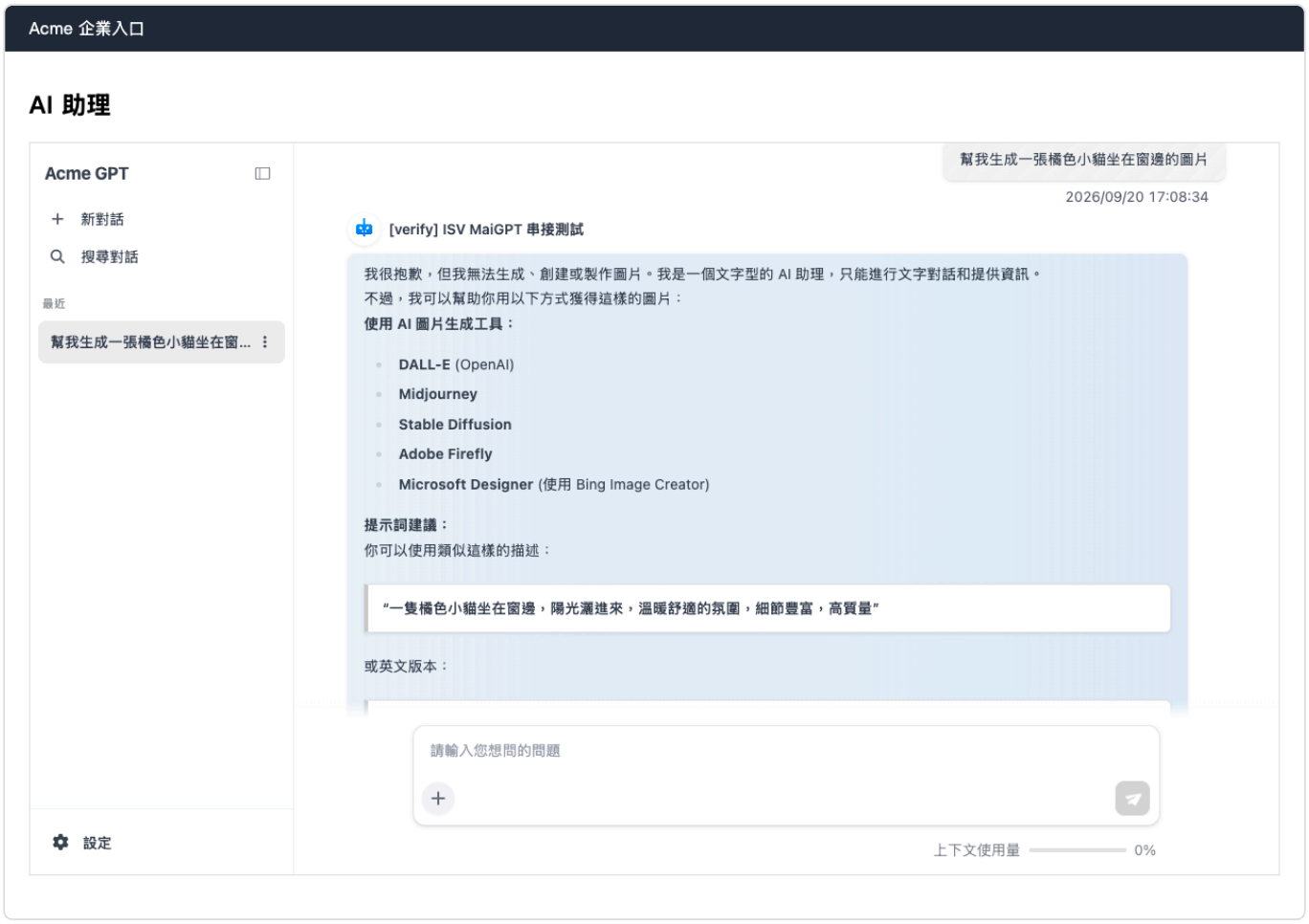
```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<webChatId created in section 2>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

To let the Agent identify signed-in users and call your APIs with their permissions, add `contactId` or `auth` as described in the Software Vendor Integration Guide.

Verification: The Same Embed Code, Before and After Adding Capabilities

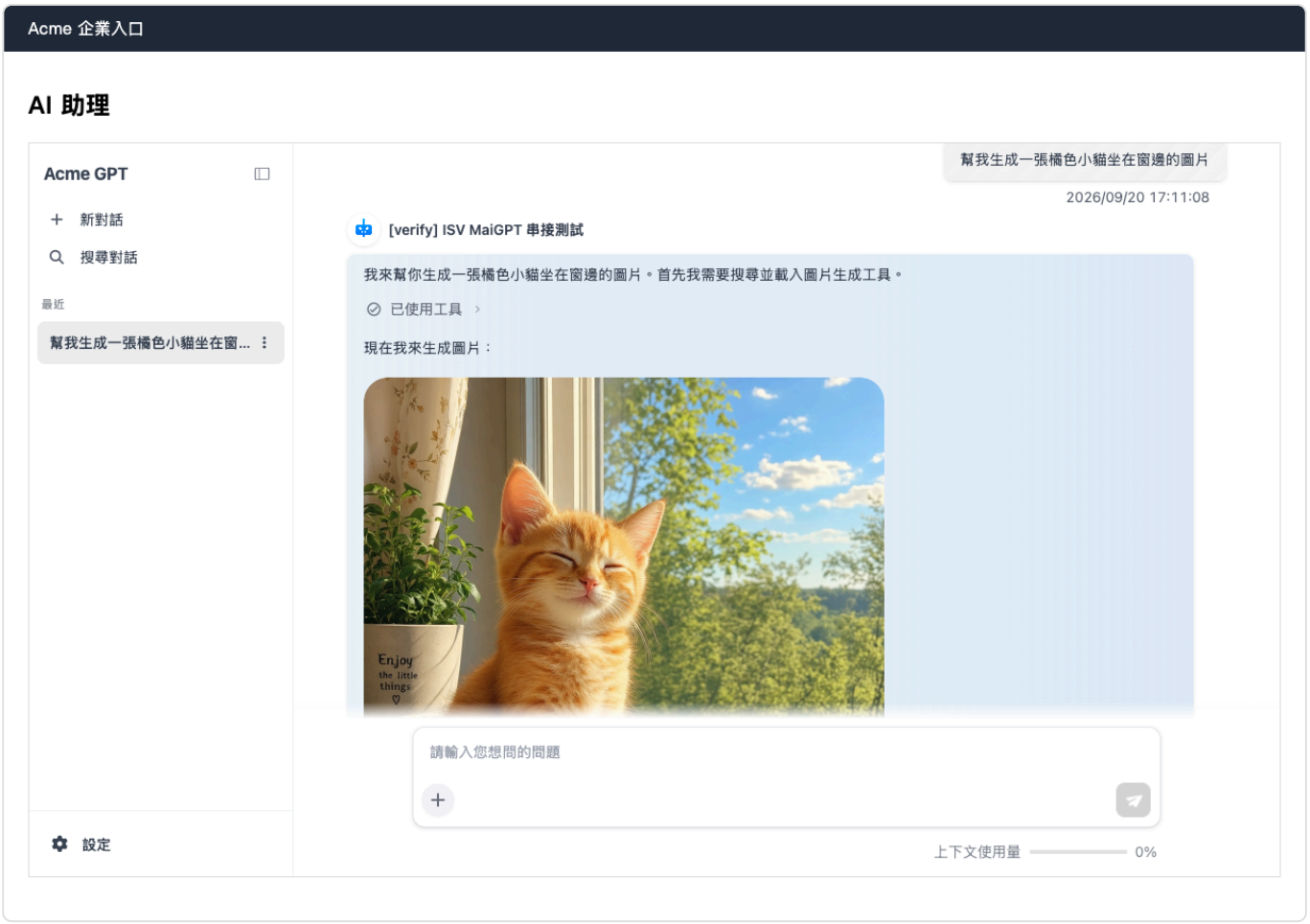
The two screenshots below show the same AI Agent and the same embed code. Only the “reply mode + tools” settings differ:

Before: RAG Q&A mode, no tools



The interface uses MaiGPT window mode, but the Agent replies “I cannot generate, create, or produce images”

After: Agent mode + image generation tool



With the same embed code, the Agent calls the image generation tool and produces an image directly in the conversation

4. When to Use the MaiGPT Product Itself

If you want “an internal Agent with ChatGPT-level capabilities for our employees”, rather than “an embedded experience in our product for our customers”, using the MaiGPT product directly is the easiest option:

Employees can sign in to the platform as organization members and use it via the web version, iOS App, LINE, or Microsoft Teams

Image generation, Deep Research, connectors, and skills are built in. Administrators only need to use role usage permissions to decide who can use which features

To add company-specific knowledge, make the knowledge bases available within the scope visible to members; you do not need to create another Agent

“Embed it in our system for organization members” → use embedding with member identity in section 5. If any of these apply— “for non-members (customers, registered users)”, “add our own API tools”, or “change instructions or models” —use the custom Agent approach in sections 2 and 3.

5. Embed with Member Identity: Embed the MaiGPT Product in Your System for Organization Members

> For a walkthrough from start to finish, see Add MaiGPT to Your System. This section provides a complete parameter reference.

Use this approach when your system's users **are also members of a MaiAgent organization** (such as an enterprise portal, employee workspace, or ISV-managed internal enterprise system) and you want the MaiGPT product itself without creating another Agent. **Your backend** uses the organization API Key to issue a short-lived token for the signed-in member. The frontend embeds MaiGPT's Web Chat using the SDK's existing member identity injection settings.

sequenceDiagram participant U as Member browser participant P as Your backend participant M as MaiAgent API participant W as MaiGPT Web Chat U->>P: Signed in, opens a page containing MaiGPT U->>P: SDK calls auth.getToken → requests your backend P->>M: POST /api/v1/members/embed-tokens/ (Api-Key, specifying member email) M-->>P: accessToken (15 minutes), organizationId, member.userId P-->>U: Returns to getToken U->>W: SDK creates conversations and sends messages as the member Note over U,W: When the token is near expiry, iframe notifies SDK, which calls getToken again to renew automatically

Mermaid 圖表

Prerequisites

An **organization API Key** whose holder is the organization owner or has the “Organization Overview” management permission through their role—only a Key issued by someone who can manage members can issue tokens on members' behalf. For obtaining an API Key, see Quickstart

The user must already be a member of the organization, with a role that permits MaiGPT access (MaiGPT Role Usage Permissions). Token issuance returns 404 if the member does not exist; it does not provision members automatically. For how users become members (SCIM provisioning API, admin console invitations, SSO), see Member Provisioning. The standard ISV-managed integration uses SCIM to create members, then issues tokens as described in this section

MaiGPT's Web Chat ID: the entry returned by `GET /api/v1/web-chats/` with the name `MaiGPT` and `accessType` set to `web`

Step 1: Issue a Member Token on Your Backend

Create an endpoint on your backend that only signed-in users can call (for example, `GET /maigpt-token`). It calls MaiAgent internally, and the frontend SDK calls it when needed:

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
  -H "Authorization: Api-Key <your organization API Key>"
  -H "Content-Type: application/json"
  -d '{"email": "alice@acme.com"}'
```

Provide either `email` or `memberId` (`memberId` is the MaiAgent member ID). Success returns `201` (`expiresAt` is in epoch milliseconds):

```
{
  "accessToken": "<short-lived member JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": {
    "id": "c1a4 ... - ... ",
    "userId": "5e77 ... - ... ",
    "email": "alice@acme.com",
    "name": "Alice"
  }
}
```

Status code	Meaning
`400`	Both `email` and `memberId` were provided, or neither was provided
`401`	The request did not authenticate with Api-Key (member sign-in tokens and anonymous requests are rejected)
`403`	The API Key holder is neither the organization owner nor authorized to manage the organization
`404`	The specified member does not exist, is disabled, or does not belong to the Key's organization

> Token rules: tokens have a fixed lifetime of 15 minutes, with no refresh token and no option to extend them. You can issue multiple tokens for the same member; previous tokens remain valid until expiry. This token can only be used for Web Chat conversations (the three calls for creating a conversation, sending messages, and continuing after handoff). Any /api/v1/ endpoint, other conversation management operation, or WebSocket request using it returns 401. Each issuance is recorded in the organization's API request logs for auditing (the token in the response is not logged).

Step 2: Give the Frontend SDK a Token Retrieval Function

Use the same embed settings as section 3, with an additional `auth.getToken`, and replace `webChatId` with MaiGPT's Web Chat ID:

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    auth: {
      // Called once when the SDK initializes, then again near token expiry to
renew automatically; no timer is needed
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' })  //
Your backend; see step 1
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

Fields in the object returned by `getToken`:

Field	Required	Description
`accessToken`	✅	`accessToken` from the step 1 response
`organizationId`	✅	`organizationId` from the step 1 response
`sourceId`	Recommended	`member.userId` from the step 1 response; you can also set it in `auth.sourceId`. If both are provided, `auth.sourceId` takes precedence

Place `auth` inside `maiagentChatbotConfig` so the SDK knows to wait for member identity injection before creating the first conversation when it opens the chat window. If you call `MaiAgent.auth.setup()` only after the SDK loads, it may attempt to create the first conversation anonymously before identity injection and be rejected.

> sourceId must be member.userId from the response, not member.id or the user ID in your system. MaiGPT uses it to associate conversations with the member. An incorrect value creates an anonymous conversation and prevents access to that member's MaiGPT history.

How Expiry and Renewal Work

Tokens last 15 minutes. The iframe notifies the SDK 60 seconds before expiry. The SDK calls `getToken` again to obtain and inject a new token. **The same conversation continues without reloading the page or iframe**, transparently to the user.

Every `getToken` call must request a new token from MaiAgent; never return a cached value: the SDK schedules the next renewal based on the token's own lifetime. Returning the same token as before is treated as a failure and triggers the retry process below.

If `getToken` fails (for example, the user has signed out or your backend returns 401), the SDK retries after 30, 60, and 120 seconds. If all retries fail, it stops and logs an error in the console. After the token expires, the next message fails to send and displays the existing error prompt. The conversation and messages already sent remain available.

When users sign out of your system, call `MaiAgent.auth.signOut()`. The SDK clears their identity and stops renewal; issued tokens remain valid for at most another 15 minutes. When another user signs in, call `MaiAgent.auth.setup()` again. Any in-progress renewal for the previous user is invalidated.

If the Web Chat enables Source ID Signature Verification, signatures are single-use. The SDK does not reuse the initial `ts`
`sig` during renewal. Have `getToken` return a new signature as well (`{ accessToken, ts, sig }`).

If you provide a static `auth.accessToken` instead of using `getToken`, the SDK does not renew it. After 15 minutes, you must call `MaiAgent.auth.setup()` again yourself.

Choosing Between This Approach and Your Own Agent

	Embed MaiGPT with member identity (this section)	**Your own Agent + MaiGPT window mode (sections 2 and 3)**
Users	Must be organization members	Anyone (customers, registered users, anonymous visitors)
Capabilities	Full MaiGPT suite: image generation, Deep Research, connectors, skills, subject to role permissions	The tools and knowledge bases you select
Custom instructions/models	No (centrally configured by the platform)	Yes
Backend work	One endpoint that calls the token issuance API	None (optional contact identity synchronization)
Billing attribution	The member's credits	The Agent/Web Chat

6. Frequently Asked Questions

▶ I see a Web Chat named MaiGPT under “Conversation Platforms” in the admin console. Can I use its embed code?

▶ Can my own Agent have the image generation, Deep Research, and connectors available in the MaiGPT product?

▶ The reply mode is already Agent and an image generation tool is attached. Why does it still not generate images?

▶ How does MaiGPT window mode differ from floating/sidebar?

MaiAgent Serve｜Customer Service Integration

Software Vendor Integration Guide: Embed AI Agents for Your Customers

This page is for software vendors embedding MaiAgent Web Chat in their own products and integrating permissions with their account systems. After users log in to your system, the AI Agent recognizes them, retrieves their conversation history, and calls your system's APIs with **their permissions**.

The integration journey has five stages. This page explains what to do at each stage and in what order, with links to the relevant technical documentation for details.

Integration Overview

flowchart LR subgraph ONCE["One-time setup"] A["Embed Web Chat"] --> B["Backfill existing users in bulk"] end subgraph DAILY["Daily operations (account lifecycle)"] C["Create a contact when adding a user"] --> D["Update Token at login"] D --> E["Synchronize again on Token refresh"] E --> F["Revoke credentials at logout"] end ONCE --> DAILY

Mermaid 圖表

> The users on this page are your customers (mapped to MaiAgent contacts). If you are integrating your company's or your customer's employees so they become MaiAgent organization members (using MaiGPT, role permissions, and Credits quotas), see Member Provisioning: SCIM
> Invitations
> SSO.

There is one core concept: **Contact**. Each user in your system maps to a MaiAgent contact. A contact holds conversation history, personalized attributes, knowledge base query scope, and tool credentials. Every permission integration action maintains this mapping between your users and contacts and their credentials. For an introduction, see Contacts: Overview and Integration.

Frontend and Backend Responsibilities

Throughout the journey, your frontend only loads the chat window. Your backend initiates all calls containing a Token or API Key:

flowchart LR subgraph FE["Your frontend"] A["Embed code loads SDK
config includes contactId"] B["Call signOut at logout"] end subgraph BE["Your backend"]
C["Login/refresh: POST setup-contact-credentials"] D["Bulk backfill: POST bulk-import"] E["Data
synchronization: PATCH contacts"] F["Logout: DELETE tool credentials"] end subgraph
MA["MaiAgent"] G["Chat window
chat.maiagent.ai"] H["API
api.maiagent.ai"] end A --> G B --> G C --> H D --> H E --> H F --> H

Mermaid 圖表

`setup-contact-credentials` requires no API Key (it is a public endpoint). `bulk-import`, `PATCH`, and `DELETE` require `Authorization: Api-Key` authentication and must be called from the backend.

Step 1: Embed Web Chat

First decide which AI Agent to embed. Embedded capabilities (image generation, web access, and tool calls) depend on the Agent configuration behind Web Chat, rather than the embedding method. The organization's built-in MaiGPT product is available only to logged-in members, and its Web Chat cannot be embedded in external systems. Create an AI Agent for your product, set it to Agent mode, and attach the required tools. See MaiGPT Integration: Choose the Right Agent and Add Capabilities.

Choose the embedding method that fits your product. Both use the same SDK and contact mechanism:

Form	Suitable for	Documentation
Chat bubble (floating/sidebar)	Supporting features such as customer service and consultation	Web Chat Embedding and SDK
MaiGPT mode (a full ChatGPT-like interface)	The main feature area of a page or a site-wide AI entry point	Embedding in MaiGPT Mode

For Web Chat with identity integration, configure the Allowed Embedding Domains list at this stage (administrators enter it in the conversation platform's basic settings). This limits origins that can embed Web Chat and call the identity synchronization API. See Allowed Embedding Domains.

Step 2: Initial Integration—Backfill Existing Users in Bulk

When integrating an existing system for the first time, backfill contacts for existing users first. There are two options:

Method	Suitable for	Documentation
Excel bulk import (`POST /contacts/bulk-import/`)	Large numbers of users (up to 10,000 rows per request); can also import knowledge query scope and API tool credentials	Bulk Contact Import
Call the identity synchronization API for each user	Gradual backfilling through the login flow, or small numbers of users	Backfilling Existing Users in Bulk

You do not need users' Access Tokens at backfill time. Create the mappings first; the login flow in Step 4 adds Token credentials the next time each user logs in.

Step 3: Create a Contact When Adding a User

After backfilling, add an API call to your system's account creation flow so each new user has a corresponding contact from the start:

Call the Identity Synchronization API (`setup-contact-credentials`) from the backend, using your system's user identifier as `sourceId`

Store the returned `contactId` in your member table (for the recommended field design, see Where to Integrate in the Login Flow)

`sourceId` is an idempotency key: repeated calls with the same `sourceId` only update the contact without creating duplicates. This step can safely coexist with Steps 2 and 4.

Step 4: Login and Token Refresh

After each successful login generates an Access Token, the backend calls the same identity synchronization API again with the **new Token** (`mcpCredentials`). This allows the AI Agent to call your system's APIs with that user's identity and permissions. Use the same approach on Token refresh: call the API again. For the same contact and tool pair, it updates credentials rather than creating duplicates. For the full sequence diagram and considerations (asynchronous calls and failures that do not block login), see Contact Identity Synchronization and Token Updates.

The frontend passes `contactId` when initializing Web Chat on the page. See Identify Users.

> If you only need the AI to recognize users without calling your system's APIs, you can skip the backend entirely: include auth in the frontend config. For the tradeoffs, see Choosing Frontend auth or Backend Integration.

Step 5: Logout and Credential Revocation

At logout, take two actions with different scopes. We recommend both:

Frontend: Call the SDK's `signOut()` to return the chat window to an anonymous state

Backend: Delete the contact's tool credentials so the AI immediately loses the ability to call tools as that user

For API details and a comparison of the two actions, see Logout and Credential Revocation.

Integration Checklist

Stage	Action	Documentation
Setup	Create your own AI Agent: Agent mode + tools such as image generation + Web Chat	MaiGPT Integration: Choose the Right Agent and Add Capabilities
Setup	Embed Web Chat (standard or MaiGPT mode)	Web Chat Embedding and SDK MaiGPT Mode
Setup	Enter Allowed Embedding Domains in the conversation platform's basic settings	Allowed Embedding Domains
Setup	Add a `maiagent_contact_id` field to the member table	Where to Integrate in the Login Flow
Setup	Backfill existing users in bulk	Bulk Contact Import
Operations	Call the identity synchronization API in the account creation flow	Contact Identity Synchronization and Token Updates
Operations	Update credentials in the login/Token refresh flow	When to Call
Operations	Revoke credentials in the logout flow	Logout and Credential Revocation
Advanced	Synchronize personalized attributes such as department and membership tier	Synchronize Contact Data
Advanced	Use query metadata to limit each user's knowledge base retrieval scope	Knowledge Management Permissions (Query Metadata)

Web Chat Embedding and SDK

Add an embed snippet to your web page to bring MaiAgent's AI Agent into your website. Once the snippet loads `embed.min.js` (referred to below as the SDK), the SDK creates a chat button and an iframe chat window on the page, and exposes the global `MaiAgent` JavaScript API for programmatic control.

This page is the complete technical reference for embedding Web Chat. Related topics:

Integrating MaiGPT: Choose the Right Agent and Enable Its Capabilities — The MaiGPT product vs. MaiGPT window mode, and how to enable image generation and tools for the embedded Agent

Embedding Web Chat in MaiGPT Mode — A full-featured, ChatGPT-like conversation interface

Injecting Page Context into Web Chat — Inject page information into the LLM System Prompt

Contact Identity Synchronization and Token Updates — Create contacts and credentials through your backend

Software Vendor Integration Guide — The complete embedding and permission integration journey (batch provisioning, login synchronization, and credential revocation on logout)

1. Quick Start

Add the following before `</body>` on your page:

```
<script>
  window.maiagentChatbotConfig = {
    webChatId: 'your-web-chat-id',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    primaryColor: '#1890ff',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

Get `webChatId` from the Web Chat settings page in the admin console (see Connecting Conversation Platforms: Website; the embed code generated by the console includes it automatically)

The `baseUrl` and SDK URL shown here use the SaaS environment (`chat.maiagent.ai`); for private cloud or on-premises deployments, replace them with your environment's domain

Define `window.maiagentChatbotConfig` before the SDK script loads

Default behavior: A chat button appears in the bottom-right corner of the page. Clicking it opens a floating chat window, and users can switch between floating and sidebar window modes.

Initialization Timing

After loading, the SDK registers a `document.body.onload` handler to initialize itself. Manual intervention is usually unnecessary. To control initialization timing (for example, to wait for a container to mount in an SPA or for delayed rendering):

Load the script later: Dynamically insert the